SQL — Exhaustive Syntaxes, Data Types & Internals (PostgreSQL)

Contents

> *[!NOTE]* ***How to use these notes.*** *A from-scratch, exhaustive reference for SQL using the* ***PostgreSQL*** *dialect. Every statement, clause, operator, and data type is explained from first principles — not just shown. Heavy concepts carry a "Heavy Concept" banner and are taught with an analogy, a concrete example, then the precise technical rule.*

> *[!NOTE]* ***Prerequisites.*** - Assumed (not re-taught): *the relational model, ER modeling (entities, relationships, cardinality, keys), and normalization (1NF–3NF/BCNF, functional dependencies).*

> - Taught here from zero: *all SQL syntax (every statement, clause, and operator), transactions and isolation, query execution, and indexing — no prior SQL assumed.* - Dialect: *all examples are **PostgreSQL**. SQL is an ISO standard with vendor dialects; this document commits to Postgres rather than comparing engines.*

1 1. Orientation & RDBMS Basics

SQL (Structured Query Language) is the standard language for communicating with Relational Database Management Systems (RDBMS). Before diving into the exhaustive syntaxes and types, we must establish what we are querying and how the language is standardized.

1.1 1.1 The Relational Model

The relational model, introduced by Edgar F. Codd in 1970 [1], revolutionized data storage by separating the *logical* structure of data from its *physical* storage.

As a brief recap of the core terminology: a **relation** is a two-dimensional table, a **tuple** is a single row, an **attribute** is a column, and a **key** uniquely identifies a row or links tables together.

> *[!NOTE] **The Information Rule:** Codd's first rule states that* all *information in a relational database must be represented explicitly at the logical level and in exactly one way: as values in tables [2].*

1.2 1.2 ISO/IEC 9075 Standard vs. Vendor Dialects

When we talk about "SQL", we are referring to a language defined by an international standard: **ISO/IEC 9075** [3]. This massive standard is divided into multiple parts, dictating core syntax like querying data or creating tables.

However, if you write SQL for PostgreSQL, you are writing a **dialect**.

Why dialects exist: While the ISO/IEC standard dictates the core syntax, database vendors introduce proprietary extensions to improve performance or add specialized features. SQL is a standard, but PostgreSQL, MySQL, and SQL Server each implement dialects with their own unique behaviors. For example, procedural extensions like PL/pgSQL (in PostgreSQL) or T-SQL (in SQL Server) go beyond the core standard to offer loops, variables, and conditional logic. Going forward, this document focuses exclusively on the **PostgreSQL** dialect.

> *[!CAUTION] **Trap: Assuming 100% Portability** A common beginner mistake is assuming a complex SQL query written for one database engine will run perfectly on another. While the core SQL standard is highly portable, functions for string manipulation, date math, and window function support often vary drastically between dialects.*

Failure case: This portability breaks when you migrate an application between database engines and find that your specific date-math functions or data types are not supported on the new engine.

1.2.1 References

1. E.F. Codd's Relational Model - Wikipedia - https://en.wikipedia.org/wiki/Relational_model
2. Codd's 12 Rules - GeeksforGeeks - https://www.geeksforgeeks.org/codds-12-rules-for-rdbms/
3. SQL Standard - ISO - https://www.iso.org/standard/63555.html

2 2. Exhaustive Data Types

While the ISO/IEC 9075 SQL standard defines a core set of data types, the reality of database engineering is that every major RDBMS implements them slightly differently [1]. To survive a deep technical grilling, you must know not just the standard types, but the *nuances* and traps of how PostgreSQL actually stores data.

2.1 2.1 Numeric Types — Complete Reference

2.1.1 Exact Numerics (Integers)

Integers are whole numbers without a fractional part. They are the simplest and most efficient way to store counts, identifiers, or mathematical whole values. Think of them as physical tally marks—you can have 1, 2, or 3 apples, but never 2.5.

PostgreSQL offers three main integer types, chosen based on the maximum value you expect to store [3]:

Type	Bytes	Range	Use Case
`SMALLINT`	2	−32,768 to 32,767	Legacy systems, strict memory constraints
`INTEGER`	4	−2.1 billion to 2.1 billion	Default choice for most general numbers and identifiers
`BIGINT`	8	−9.2 quintillion to 9.2 quintillion	Extremely large identifiers, high-volume metrics

In SQL, to define a column as an integer, you use the type name directly.

When you need an integer that automatically increments with each new row (which is standard for creating unique identifiers, or Primary Keys), PostgreSQL uses a special keyword construct: `GENERATED ALWAYS AS IDENTITY`. This tells the database engine to automatically calculate and

assign the next available integer whenever a new row is added, guaranteeing uniqueness without you having to track the previous highest number.

```
-- Creating a table (a collection of rows) with an auto-incrementing integer
column
CREATE TABLE users (
    user_id INTEGER GENERATED ALWAYS AS IDENTITY,
    age SMALLINT
);
```

First-use glosses: - `CREATE TABLE`: A command that defines a new empty structure to hold data rows, specifying its columns and their data types. - `NULL`: In SQL, `NULL` represents a completely missing or unknown value. It is not zero; it is not an empty string. It means "absence of data." If you try to add an integer to `NULL`, the result is `NULL`.

Failure case: If you define an identifier column as `SMALLINT` and your system creates its 32,768th user, the database will throw an overflow error and completely halt new registrations. Always default to `INTEGER` or `BIGINT` for identifiers unless you have a proven, extreme memory constraint.

2.1.2 Exact Numerics (Fixed-Point Decimals)

> *[!TIP]* ***Heavy Concept: Fixed-Point Decimals (`DECIMAL` / `NUMERIC`)***

Analogy: Think of a standard integer like counting whole apples (1, 2, 3). Think of floating-point math like guessing the weight of those apples on a loose spring scale—it's fast and close enough, but imprecise. Fixed-point decimal types are like an accountant's physical ledger book: every single cent is perfectly tracked in its own rigid column, and rounding never happens unless you explicitly ask for it.

Diagram-in-words: Imagine a rigid grid with exactly 10 slots for digits, and a permanent decimal point bolted firmly in place after the 8th slot. You have exactly 8 digits for the dollars, and exactly 2 digits for the cents. The decimal point never "floats" or moves.

Concrete Example: If an item costs $12.34, you want it stored exactly as 12.34. If you used a floating-point type instead, the computer's base-2 memory might store it as 12.340000000000001 or 12.339999999999999. Over millions of transactions, these microscopic errors compound into real missing money.

Plain-English: A fixed-point decimal type guarantees that the exact fractional number you insert is exactly the number you read back. It trades computational speed and storage space to ensure absolute mathematical precision.

Technical: In PostgreSQL, `DECIMAL` and `NUMERIC` are perfectly equivalent. You define them using the syntax `NUMERIC(p, s)`. - **Precision (p):** The total maximum number of significant digits across the entire number (both left and right of the decimal). - **Scale (s):** The exact number of digits to the right of the decimal point.

For example, `NUMERIC(5, 2)` can store any value from −999.99 to 999.99.

Type	Syntax	Use Case
`DECIMAL(p,s)`	`DECIMAL(10,2)`	Currency, financial data, exact scientific measurements
`NUMERIC(p,s)`	`NUMERIC(5,3)`	Identical to DECIMAL

Why it matters here: Financial applications mandate exact math.

> *[!CAUTION]* ***Trap: Using FLOAT for Currency*** *A classic beginner mistake is storing money as a floating-point number [2].* Failure case: *This breaks when you sum thousands of financial transactions and find your balance sheet is off by a few cents due to floating-point representation limits in binary memory. Always use* `DECIMAL` *or* `NUMERIC` *for currency.*

2.1.3 Approximate Numerics (Floating-Point)

Unlike fixed-point decimals, floating-point types allow the decimal point to "float" dynamically based on the size of the number. They trade absolute precision for the ability to represent an astronomically wide range of values in a very small amount of memory.

Type	Bytes	Precision	When to Use
`REAL`	4	~6 decimal digits	Scientific data, sensor readings
`DOUBLE PRECISION`	8	~15 decimal digits	Complex scientific computations

Failure case: If you store `123456789.12` in a 4-byte `REAL` column, it may read back as `123456792.00` because you exceeded its ~6 digits of guaranteed precision.

2.2 2.2 Character Strings — Complete Reference

Text data in SQL is referred to as a "character string" or just "string". PostgreSQL provides different ways to store text depending on whether the length is predictable.

> *[!TIP]* ***Heavy Concept: Variable vs. Fixed-Length Strings***

Analogy: A fixed-length string (`CHAR`) is like a physical hotel registration book where every name gets exactly a 50-character wide box. If a guest's name is "Bob" (3 letters), the hotel clerk fills the remaining 47 spaces with blank whiteout. A variable-length string (`VARCHAR` or `TEXT`) is like

a dynamic digital sticky note: it only takes up exactly as much space as the letters written on it, plus a tiny tag recording how long the word is.

Concrete Example: If you store the state code 'CA' in a `CHAR(2)` column, it takes exactly 2 characters of space. If you store 'CA' in a `CHAR(10)` column, PostgreSQL pads it with 8 invisible spaces to make it exactly 10 characters long ('CA').

Plain-English: Fixed-length types force every entry to be exactly the same size, padding shorter entries with spaces. Variable-length types adapt to the data, storing only the characters provided.

Technical: PostgreSQL natively stores all text using UTF-8 encoding. You define string columns using:

Type	Behavior	Max Length	Trailing Spaces
`CHAR(n)`	Fixed-length, space-padded	10,485,760 bytes	Padded with spaces
`VARCHAR(n)`	Variable-length with a strict limit	10,485,760 bytes	Not padded
`TEXT`	Unlimited variable-length	~1 GB	Not padded

```
-- Creating a table with different character string types
CREATE TABLE user_profiles (
    state_code  CHAR(2),        -- Always exactly 2 chars. Good for US
states ('CA', 'NY').
    username    VARCHAR(50),    -- Variable length, but strictly capped at
50 chars.
    biography   TEXT            -- Variable length, functionally unlimited.
Good for essays.
);
```

Alternative rejected: Why not use `CHAR(n)` for everything to keep rows uniform? Because space-padding wastes massive amounts of disk space for data that varies in length (like names or emails), and it requires you to actively trim off the trailing spaces later when querying the data.

Failure case: If you store an email address in a `CHAR(255)` column, searching for `'user@example.com'` might fail unless you account for the 239 invisible spaces padded at the end. In PostgreSQL, always default to `TEXT` or `VARCHAR` unless the data has a globally standardized fixed length (like a 2-letter country code).

2.3 2.3 Date, Time, Timestamp, and Interval — Complete Reference

Storing temporal (time-based) data correctly is one of the hardest parts of database design because of timezones, leap years, and daylight saving time.

Type	Behavior	Example Format
`DATE`	A calendar date with no time of day	'2025-06-27'
`TIME`	A time of day with no date attached	'17:30:00'
`TIMESTAMP`	Date and time, *without* timezone awareness	'2025-06-27 17:30:00'
`TIMESTAMPTZ`	Date and time, *with* timezone awareness	'2025-06-27 17:30:00+05:30'
`INTERVAL`	A span of time, not tied to a specific calendar date	'3 days 2 hours'

In SQL, a literal time value is written as a string inside single quotes, prefixed by the keyword of its data type.

```
-- Date and time literals (how you write them explicitly in a query)
SELECT DATE '2025-06-27';
SELECT TIME '17:30:00';
SELECT TIMESTAMP '2025-06-27 17:30:00';
SELECT TIMESTAMPTZ '2025-06-27 17:30:00+05:30';
SELECT INTERVAL '3 days 2 hours';

-- Date arithmetic: You can add an INTERVAL to a DATE to get a new date in
the future
SELECT CURRENT_DATE + INTERVAL '30 days';
```

First-use gloss: `SELECT`: The fundamental SQL command used to ask the database to return data to you. When used without a table, it simply evaluates and returns the expression provided.

> *[!TIP]* ***Heavy Concept: Timezone Awareness (`TIMESTAMPTZ`)***

Analogy: If you tell a friend "Let's talk at 3:00 PM," that works if you are in the same room. But if they fly to Tokyo and you stay in New York, "3:00 PM" becomes dangerously ambiguous. You need an absolute, globally agreed-upon anchor, like "3:00 PM New York Time."

Concrete Example: A user clicks a button at exactly 17:30:00 in India (UTC+05:30). If you store this in a plain `TIMESTAMP` column as just `2025-06-27 17:30:00`, the database has no idea where on Earth that happened. When an analyst in New York queries that data, they might falsely assume it happened at 17:30 New York time.

Plain-English: The `TIMESTAMPTZ` (Timestamp with Time Zone) type converts the incoming time to UTC (Coordinated Universal Time) for storage. When you ask the database for that value later, it automatically translates the stored UTC time back into your local timezone. It represents a specific, absolute moment in the history of the universe.

Technical: `TIMESTAMPTZ` does not actually store the original timezone string you provide. It reads your timezone offset (`+05:30`), calculates what the equivalent UTC time is, stores *that* UTC integer on disk, and discards your original timezone label.

> *[!CAUTION]* ***Trap: Timezone Ignorance*** *Storing a real-world event in a plain `TIMESTAMP` without knowing the application's timezone means you permanently lose the absolute point in time.* Alternative rejected: *Just storing a Unix epoch (an integer counting seconds since 1970) avoids timezone bugs, but makes querying for "all events that happened on a Tuesday" computationally expensive because the database has to convert millions of integers back to calendar days on the fly. Use `TIMESTAMPTZ` for events.*

2.4 2.4 Binary and Boolean Types

2.4.1 Boolean

A boolean represents a simple truth value.

Type	Behavior	Valid Inputs
`BOOLEAN`	Represents truth states	`TRUE`, `FALSE`, `NULL`

In PostgreSQL, the `BOOLEAN` (or `BOOL`) type is deeply integrated [3]. Unlike some other dialects that fake booleans using integers (where 1 is true and 0 is false), PostgreSQL treats true and false as native, distinct concepts.

2.4.2 Binary Data (`BYTEA`)

Sometimes you need to store raw, unformatted computer data—like a compiled executable, a compressed ZIP file, or an encrypted password hash—where text encoding rules like UTF-8 would corrupt the bytes.

Type	Use Case
`BYTEA`	Raw binary data (images, encrypted blobs, file contents)

Why it matters here: If you try to store a raw JPEG image in a `TEXT` column, PostgreSQL will attempt to validate it as UTF-8 text, fail, and reject the data. `BYTEA` tells the database to store the exact binary zeroes and ones without trying to understand them.

2.5 2.5 Advanced Types (JSON, Arrays, UUID, Enum, XML, Range)

PostgreSQL shines in its support for complex, non-traditional relational data types.

> *[!TIP]* ***Heavy Concept: JSON vs JSONB***

Analogy: Storing plain `JSON` is like keeping a paper document in a filing cabinet; if you want to find a specific word on page 3, you have to pull out the whole document and read it from top to bottom every time. Storing `JSONB` (Binary JSON) is like scanning that document into a searchable database; the initial scan takes a bit longer, but finding that word later is instantaneous.

Concrete Example: You receive a large JSON payload from an external API: `{"user": "alice", "clicks": 42}`.

Plain-English: The `JSON` type stores exactly the text string you gave it, including all spaces and duplicate keys. To query it, the database must parse the text on every single query. The `JSONB` type parses the text *once* when you insert it, removing unnecessary spaces and deduplicating keys, and stores it in a custom binary format [4].

Technical: `JSONB` supports advanced indexing (like GIN indexes), which allows PostgreSQL to look deep inside the nested JSON structure instantly, without scanning the whole table. Always use `JSONB` unless you have a strict legal requirement to preserve the exact whitespace of an incoming text payload.

```
-- Creating a table with a JSONB column
CREATE TABLE web_events (
    event_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    event_payload JSONB
);

-- Querying deep inside the JSONB structure using specialized operators
-- The ->> operator extracts a nested JSON value as plain SQL text
SELECT event_payload->>'user_name' FROM web_events;

-- The @> operator checks if the left JSON payload contains the right JSON
payload
SELECT * FROM web_events WHERE event_payload @> '{"action": "click"}';
```

2.5.1 Arrays

Relational databases traditionally require you to put multiple values into a separate, linked table. PostgreSQL allows you to store an array (a list) of values directly inside a single column.

```
CREATE TABLE survey_responses (
    response_id INTEGER PRIMARY KEY,
    selected_tags TEXT[],       -- An array containing multiple TEXT values
    quiz_scores INTEGER[]       -- An array containing multiple INTEGER
values
);

-- The ARRAY keyword defines the list of items
INSERT INTO survey_responses (response_id, selected_tags, quiz_scores)
VALUES (1, ARRAY['sql', 'databases'], ARRAY[85, 92, 78]);

-- The ANY() operator checks if a specific value exists anywhere inside the
array
SELECT * FROM survey_responses WHERE 'sql' = ANY(selected_tags);
```

Alternative rejected: Arrays violate the strict rules of database normalization (specifically First Normal Form, which dictates atomic values). However, for simple lists like "tags" where you never need to join the tags to another table, arrays offer immense performance benefits over creating complex join structures.

2.5.2 UUID (Universally Unique Identifier)

A UUID is a 128-bit number formatted as a 36-character string (e.g., `550e8400-e29b-41d4-a716-446655440000`). It is used when you need to generate a unique identifier without relying on a central database counter.

```
CREATE TABLE distributed_nodes (
    -- gen_random_uuid() is a built-in function that creates a secure,
random UUID
    node_id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    location VARCHAR(100)
);
```

2.5.3 Enum Types

An Enum (Enumeration) is a static, ordered set of specific text values. It acts as a strict whitelist.

```
-- First, define the new custom type and its allowed values
CREATE TYPE shirt_size AS ENUM ('XS', 'S', 'M', 'L', 'XL', 'XXL');

-- Then use it in a table
CREATE TABLE products (
    product_id INTEGER,
```

```
    size shirt_size
);

-- This succeeds because 'M' is in the whitelist
INSERT INTO products VALUES (1, 'M');

-- This fails instantly and protects your data integrity
INSERT INTO products VALUES (2, 'XXXL');
```

2.5.4 Range Types

Range types represent an upper and lower bound of values in a single column. They are incredibly powerful for scheduling systems.

```
CREATE TABLE meeting_rooms (
    room_number INTEGER,
    -- TSTZRANGE means Timestamp with Time Zone Range (a start and end time)
    booked_during TSTZRANGE
);

-- The @> operator checks if a specific moment falls within the stored range
SELECT * FROM meeting_rooms
WHERE booked_during @> TIMESTAMPTZ '2025-06-27 15:00+05:30';
```

2.5.5 XML

For legacy enterprise systems, PostgreSQL provides an `XML` type that checks if the incoming text is well-formed XML and provides functions (like `xpath()`) to query nodes inside it.

```
CREATE TABLE configuration_files (
    config_id INTEGER,
    settings XML
);
```

2.6 2.6 Data Type Selection Cheat Sheet

Data	Correct Type	Wrong Type	Why
Currency	`NUMERIC` / `DECIMAL`	`REAL`, `DOUBLE PRECISION`	Floating-point rounding destroys financial accuracy.
Email Address	`VARCHAR(255)` / `TEXT`	`CHAR(255)`	Wastes disk space with invisible spaces.
Unique ID	`UUID` / `BIGINT`	`VARCHAR(36)`	Storing a UUID as plain text wastes space and slows down index lookups.
Yes/No Flag	`BOOLEAN`	`VARCHAR(3)` (‘yes’/‘no’)	Semantic clarity and minimal storage footprint.
Real-World Events	`TIMESTAMPTZ`	`TIMESTAMP`, `BIGINT`	Prevents permanent loss of timezone context.
Network IP	`INET` (Postgres specific)	`VARCHAR(45)`	Built-in network types validate the IP and allow subnet querying.
Country Code	`CHAR(2)`	`VARCHAR(100)`	Enforces fixed length for standardized ISO 3166 codes.
Article Body	`TEXT`	`VARCHAR(8000)`	Arbitrary length caps lead to broken, truncated data.

2.6.1 References

1. Data Type Standards - GeeksForGeeks - https://www.geeksforgeeks.org/sql-data-types/
2. Why not use Double or Float to represent currency? - StackOverflow - https://stackoverflow.com/questions/3730019/why-not-use-double-or-float-to-represent-currency
3. PostgreSQL Data Types - Postgres Docs - https://www.postgresql.org/docs/current/datatype.html
4. PostgreSQL JSON Types - Postgres Docs - https://www.postgresql.org/docs/current/datatype-json.html

3 3. Data Definition Language (DDL)

Data Definition Language (DDL) encompasses the SQL commands used to define, modify, and destroy the structural blueprint of your database objects (tables, views, indexes, schemas) rather than the data itself [1]. Because DDL operations define the schema (the formal structure and rules of the database), they are the foundation upon which all other database operations rest.

3.1 3.1 CREATE TABLE and Constraints

The `CREATE TABLE` statement is the foundation of every database. It creates a new, empty table in the current database. However, simply defining columns and their data types is rarely enough. A relational database guarantees data integrity by enforcing rules on what data is allowed. These rules are called constraints.

Every constraint must be understood not just as syntax, but as an active guardian of data quality. If an `INSERT` or `UPDATE` operation violates a constraint, the database rejects the operation entirely (a failure case to protect the integrity of the system).

3.1.1 The Full Anatomy of CREATE TABLE

Here is the exhaustive anatomy of a table creation, using PostgreSQL syntax:

```
CREATE [TEMPORARY | TEMP] TABLE [IF NOT EXISTS] schema_name.table_name (
    -- Column definitions: name, type, and optional inline constraints
    column_name  data_type  [column_constraints],
    ...
    -- Table-level constraints (declared after all columns)
    [table_constraints]
);
```

3.1.2 Column Constraints (Inline)

Constraints can be defined inline on a specific column. This is shorthand and applies the rule directly to that single column.

- `PRIMARY KEY`: Uniquely identifies each row in a table. It is mathematically equivalent to combining a `UNIQUE` constraint and a `NOT NULL` constraint. A table can have at most one primary key.
- `NOT NULL`: Ensures that the column must always contain a value. An attempt to insert an empty (null) value breaks the rule and fails.
- `UNIQUE`: Guarantees that no two rows can share the same value in this column. (Unlike the primary key, a table can have multiple unique columns, and a unique column can often accept multiple null values since null is not equal to null).
- `CHECK`: Enforces a specific logical condition that must evaluate to true for the row to be accepted (for example, ensuring a salary is positive).

- `REFERENCES`: An inline foreign key. It dictates that the value in this column must exist in a specific column of another table.
- `DEFAULT`: Not strictly a validation constraint, but a fallback rule. If an insert operation does not provide a value for this column, the database automatically inserts the specified default value.

```
CREATE TABLE employees (
    emp_id      INT           PRIMARY KEY,                  -- PK inline
    email       VARCHAR(255)  NOT NULL UNIQUE,              -- Two
constraints applied
    first_name  VARCHAR(100)  NOT NULL,
    last_name   VARCHAR(100)  NOT NULL,
    salary      DECIMAL(10,2) NOT NULL CHECK (salary > 0),  -- Validation
rule
    dept_id     INT           REFERENCES departments(id),   -- FK inline
shorthand
    status      VARCHAR(20)   DEFAULT 'active',             -- Fallback
value
    hire_date   DATE          DEFAULT CURRENT_DATE,
    bio         TEXT,                                       -- Nullable by
default
    created_at  TIMESTAMP     DEFAULT CURRENT_TIMESTAMP
);
```

3.1.3 Table-Level Constraints

When a constraint needs to span multiple columns, or if you simply prefer to give constraints explicit names for easier debugging, you define them at the table level after all columns are listed.

```
CREATE TABLE order_items (
    order_id    INT NOT NULL,
    product_id  INT NOT NULL,
    quantity    INT NOT NULL,
    unit_price  DECIMAL(10,2) NOT NULL,

    -- Composite Primary Key: The combination of both columns must be
unique.
    CONSTRAINT pk_order_items   PRIMARY KEY (order_id, product_id),

    -- Named Foreign Keys
    CONSTRAINT fk_order         FOREIGN KEY (order_id)
                                REFERENCES orders(id)
                                ON DELETE CASCADE,

    CONSTRAINT fk_product       FOREIGN KEY (product_id)
                                REFERENCES products(id)
                                ON DELETE RESTRICT,
```

```
    -- Named CHECK constraint referencing a single column
    CONSTRAINT chk_quantity     CHECK (quantity > 0 AND quantity <= 10000),

    -- Named UNIQUE constraint spanning multiple columns
    CONSTRAINT uq_order_product UNIQUE (order_id, product_id)
);
```

3.1.4 Referential Actions for Foreign Keys (ON DELETE / ON UPDATE)

When a foreign key connects a child table to a parent table, the database must enforce consistency. But what happens if you attempt to delete or update the parent row? A referential action dictates the database's automatic response.

> *[!TIP]* ***Heavy Concept: Referential Actions Analogy:*** *Imagine a library where every Book (child) must belong to a registered Branch (parent). If the city decides to demolish a Branch, what happens to the Books? - `CASCADE`: You throw all the books in the incinerator along with the branch. (Delete the parent, automatically delete the children). - `RESTRICT`: The demolition crew arrives, sees books inside, and refuses to tear down the building. (Block the deletion of the parent entirely). - `SET NULL`: You remove the branch's name from the books, leaving them in a pile without a home. (The parent is deleted, the child's reference becomes blank).*
>
> ***Concrete Example:*** *An `orders` table references a `customers` table. If you delete customer #100, and the action is `RESTRICT`, the database returns an error: "Cannot delete customer #100, they have existing orders." If the action is `CASCADE`, customer #100 is deleted, and every order belonging to customer #100 is silently and automatically deleted as well.*
>
> ***Plain English:*** *A referential action is a contingency plan. By appending `ON DELETE` or `ON UPDATE` to a foreign key, you instruct the database engine on how to handle modifications to the parent record to prevent orphaned child records.*
>
> ***Technical (PostgreSQL Options):*** *| Action | Behavior | |—|—| | `RESTRICT` | Blocks the parent delete/update immediately if any child rows exist. (Default behavior). | | `NO ACTION` | Similar to `RESTRICT`, but deferred. The check happens at the end of the transaction, allowing intermediate steps. | | `CASCADE` | Automatically deletes (or updates) the matching child rows to follow the parent. | | `SET NULL` | Sets the foreign key column in all matching child rows to `NULL`. | | `SET DEFAULT` | Sets the foreign key column in all matching child rows to their defined `DEFAULT` value. |*
>
> ***The Trap:*** *Overusing `CASCADE` can lead to catastrophic data loss. A junior developer deletes a single company record, not realizing that `ON DELETE CASCADE` is set on departments, employees, and audit logs. The entire organization's data vanishes in milliseconds. Use `RESTRICT` by default for critical data.*

3.1.5 Auto-Incrementing Primary Keys (PostgreSQL)

You rarely want to manually generate sequential ID numbers. PostgreSQL provides mechanisms to handle this automatically. The modern SQL-standard approach is the `IDENTITY` column.

```
-- Modern standard: GENERATED ALWAYS AS IDENTITY
CREATE TABLE users (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100)
);

-- Legacy PostgreSQL approach (still very common): SERIAL
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100)
);
```

The `GENERATED ALWAYS` clause strictly prevents users from manually inserting their own values into the ID column, whereas `SERIAL` technically allows a manual override (which can cause sequence desynchronization).

3.1.6 Generated / Computed Columns

Sometimes a column's value should always be a direct mathematical calculation of other columns in the same row. A generated column automates this. In PostgreSQL, these are always `STORED` (physically written to disk).

```
CREATE TABLE line_items (
    id          INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    price       DECIMAL(10,2),
    tax_rate    DECIMAL(4,2),
    total_price DECIMAL(10,2) GENERATED ALWAYS AS (price * (1 + tax_rate))
STORED
);
```

3.1.7 Temporary Tables

A temporary table is a scratchpad. It behaves like a normal table but exists only for the duration of your current database session. Once you disconnect, the database automatically destroys the table and all its data.

```
CREATE TEMPORARY TABLE temp_scoring (
    user_id INT,
    score DECIMAL(5,2)
);
```

Failure case: If you attempt to access `temp_scoring` from a different database connection or terminal window, it will not exist.

3.1.8 CREATE TABLE ... AS (CTAS)

You can create a new table and instantly populate it with the results of a `SELECT` query. This is called a CTAS operation. It is heavily used in analytics to snapshot complex queries into physical tables.

```
CREATE TABLE high_earners AS
    SELECT emp_id, first_name, last_name, salary
    FROM employees
    WHERE salary > 100000;
```

3.2 3.2 CREATE VIEW

A View is a stored query that behaves exactly like a virtual table. It stores the SQL instructions to fetch data, but it does not store the data itself.

When you query a view, the database engine transparently executes the underlying stored query, fetches the live data from the base tables, and returns the result.

Why use views? 1. **Simplicity:** Hide complex `JOIN` logic behind a simple interface. 2. **Security:** Restrict user access to a subset of rows or columns. You can grant someone the right to read a view that excludes salary columns, without granting them access to the raw employee table.

```
CREATE OR REPLACE VIEW active_employees AS
    SELECT emp_id, first_name, last_name, dept_id
    FROM employees
    WHERE status = 'active';

-- Querying the view is identical to querying a table
SELECT * FROM active_employees WHERE dept_id = 3;
```

3.2.1 Updatable Views

In specific, simple scenarios, a view is updatable. If the view maps directly, one-to-one, back to a single underlying base table (with no aggregations like `GROUP BY`, no `JOIN`s, and no `DISTINCT` clauses), PostgreSQL allows you to run an `UPDATE` or `INSERT` against the view. The database engine will intelligently pass the modification down to the underlying table.

```
-- This UPDATE is passed through the view to modify the underlying
`employees` table:
UPDATE active_employees SET last_name = 'Doe' WHERE emp_id = 42;
```

3.2.2 Materialized Views

Unlike regular views, a Materialized View physically runs the query and stores the resulting data snapshot on disk. Because the data is pre-computed, querying a materialized view is incredibly fast.

The Trade-off: The data in a materialized view becomes stale immediately. When the base tables change, the materialized view does not automatically update. You must manually command the database to refresh the snapshot.

```
CREATE MATERIALIZED VIEW monthly_sales AS
    SELECT DATE_TRUNC('month', order_date) AS month, SUM(total) AS revenue
    FROM orders
    GROUP BY 1;

-- This requires a manual refresh to see new orders:
REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_sales;
```

3.3 3.3 CREATE INDEX

An index is a hidden data structure (usually a B-Tree) that the database builds alongside your table. Without an index, finding a specific row requires the database engine to scan every single row in the table from top to bottom (a sequential scan). With an index, the database traverses the tree structure to find the exact location of the data in milliseconds.

> *[!NOTE] Indexes are defined using DDL, but their impact is entirely on the performance of Data Manipulation Language (DML) queries. Every time you insert, update, or delete a row in the table, the database must also update the index. Therefore, over-indexing a table speeds up reading but severely slows down writing.*

```
-- Standard B-Tree index (default in PostgreSQL)
CREATE INDEX idx_emp_name ON employees (last_name);

-- Unique index (physically enforces uniqueness, identical to a UNIQUE
constraint)
CREATE UNIQUE INDEX idx_emp_email ON employees (email);

-- Composite index (an index spanning multiple columns)
-- Column order matters heavily for how the query optimizer uses it.
CREATE INDEX idx_emp_dept_salary ON employees (dept_id, salary DESC);

-- Partial index (PostgreSQL feature to index only a subset of rows)
-- This saves massive amounts of disk space if you only ever search for
active staff.
CREATE INDEX idx_active_emp ON employees (last_name) WHERE status =
'active';

-- Drop an index
DROP INDEX IF EXISTS idx_emp_name;
```

3.4 3.4 ALTER TABLE

The `ALTER TABLE` command modifies the structure of an existing table without dropping it. This allows you to evolve your schema while preserving the data inside.

```
-- Add a new column
ALTER TABLE employees ADD COLUMN middle_name VARCHAR(100);

-- Drop an existing column (destroys the data in that column)
ALTER TABLE employees DROP COLUMN middle_name;

-- Rename a column
ALTER TABLE employees RENAME COLUMN first_name TO given_name;

-- Change a column's data type
ALTER TABLE employees ALTER COLUMN salary TYPE NUMERIC(12,2);

-- Add a new constraint to an existing table
ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary > 0);

-- Drop a constraint
ALTER TABLE employees DROP CONSTRAINT chk_salary;

-- Set or Drop a column default value
ALTER TABLE employees ALTER COLUMN status SET DEFAULT 'active';
ALTER TABLE employees ALTER COLUMN status DROP DEFAULT;

-- Enforce or Remove NOT NULL rules
ALTER TABLE employees ALTER COLUMN email SET NOT NULL;
ALTER TABLE employees ALTER COLUMN bio DROP NOT NULL;

-- Rename the table itself
ALTER TABLE employees RENAME TO staff;
```

3.5 3.5 The Deletion Trap: TRUNCATE vs DROP vs DELETE

A universal concept—and frequent interview topic—is distinguishing the three ways to remove data in SQL.

> *[!CAUTION]* ***Heavy Concept: TRUNCATE vs. DELETE vs. DROP Analogy:*** *If your database table is a physical filing cabinet filled with paper folders: - `DROP TABLE` means taking the entire cabinet to the dump and crushing it. The cabinet is gone, and the files are gone. The structure no longer exists. - `DELETE FROM` means a clerk opens the cabinet, reads every single folder, removes them one by one, and painstakingly logs each removal in a ledger. The cabinet remains, the files are gone. Because of the ledger, this action is reversible if interrupted. - `TRUNCATE TABLE` means*

> *turning the cabinet upside down and shaking all the folders into an incinerator. The cabinet remains, but the files are gone instantly. The logging is minimal.*
>
> ***Concrete Example:*** *You have a `server_logs` table with 50 million rows. Running `DELETE FROM server_logs;` might take 10 minutes because the database engine writes 50 million individual deletion records to its internal transaction log. Running `TRUNCATE TABLE server_logs;` takes roughly five milliseconds.*
>
> ***Plain English:*** *`TRUNCATE` is categorized as a DDL operation, not a DML operation like `DELETE`. It operates at the storage level, deallocating the physical data pages that store the table's rows rather than evaluating and deleting rows individually.*
>
> ***The Trap:*** *Because `TRUNCATE` bypasses row-by-row deletion, it* ***cannot activate `ON DELETE` triggers****. If your application relies on a trigger firing every time a row is deleted (perhaps to update an audit table), a `TRUNCATE` operation bypasses your audit system entirely.* Note on Transactions: *In PostgreSQL, `TRUNCATE` **is** fully transactional. If you run `TRUNCATE` inside a transaction block and then issue a `ROLLBACK`, your data will be restored perfectly.*

```
-- DROP: Destroys the table structure and data entirely.
DROP TABLE IF EXISTS employees;
-- CASCADE drops objects that depend on this table (like views or foreign
keys).
DROP TABLE IF EXISTS employees CASCADE;

-- TRUNCATE: Instantly empties all rows, keeps the table structure.
TRUNCATE TABLE server_logs;
-- RESTART IDENTITY also resets any auto-incrementing sequences back to 1.
TRUNCATE TABLE server_logs RESTART IDENTITY;

-- DELETE: Row-by-row removal. It is fully logged and fires triggers.
DELETE FROM server_logs WHERE created_at < '2024-01-01';
```

3.6 3.6 Other DDL Objects in PostgreSQL

While tables are primary, PostgreSQL allows you to define other structural objects to organize and strictly type your data.

```
-- Create a Schema (a namespace or logical folder for tables)
CREATE SCHEMA IF NOT EXISTS analytics;
CREATE TABLE analytics.events (id INT, event_name TEXT);

-- Create a Sequence (an explicit standalone auto-increment generator)
CREATE SEQUENCE order_seq START WITH 1000 INCREMENT BY 1;
-- Fetching the next value advances the sequence globally.
SELECT NEXTVAL('order_seq');  -- Returns 1000, then 1001, etc.
```

```sql
-- Create a Domain (a reusable, customized data type with built-in
constraints)
CREATE DOMAIN email_address AS VARCHAR(255) CHECK (VALUE ~ '^.+@.+\..+$');
-- Now you can use `email_address` as a data type in any table.
CREATE TABLE contacts (id INT, email email_address);

-- Create a Type (an explicit enumeration of allowed string values)
CREATE TYPE mood AS ENUM ('happy', 'sad', 'neutral');
CREATE TABLE diary (id INT, feeling mood);
```

3.6.1 References

1. Data Definition Language (DDL) Overview - GeeksForGeeks - https://www.geeksforgeeks.org/sql-ddl-dql-dml-dcl-tcl-commands/
2. Difference between DELETE, DROP and TRUNCATE - StackOverflow - https://stackoverflow.com/questions/1169992/difference-between-truncate-delete-and-drop
3. PostgreSQL Official Documentation: Data Definition - PostgreSQL - https://www.postgresql.org/docs/current/ddl.html

4 4. Data Manipulation & Querying (DML & DQL)

Data Manipulation Language (DML) modifies the data within the structure (inserting, updating, deleting), while Data Query Language (DQL) retrieves it (selecting). This section is the largest because `SELECT` is the most complex and versatile statement in SQL.

4.1 4.1 The Logical Execution Order

To survive an interview grilling, you must know the difference between the *syntax order* (how you type the query) and the *logical execution order* (how the database engine actually processes it).

> *[!TIP]* ***Heavy Concept: The Logical Execution Order Analogy:*** *Imagine processing a massive stack of employee files to find the top 3 highest-paid departments. You wouldn't sort all 10,000 files first. You'd group them into department piles, calculate the total for each pile, throw out the small piles, and* then *sort the remaining handful.*
>
> *The database does the exact same thing. It processes clauses in this strict order: 1.* ***`FROM` / `JOIN`****: Identify and combine the source tables. 2.* ***`WHERE`****: Filter individual rows out immediately. 3.* ***`GROUP BY`****: Collect remaining rows into aggregate buckets. 4.* ***`HAVING`****: Filter the buckets (aggregate conditions). 5.* ***`SELECT`****: Evaluate the columns, aliases, and math to return. 6.* ***`DISTINCT`****: Remove duplicate rows from the result. 7.* ***`ORDER BY`****: Sort the final result set. 8.* ***`LIMIT` / `OFFSET`****: Truncate the output for pagination.*

> ***The Trap:*** *A classic beginner error is referencing a column alias (created in `SELECT`) inside the `WHERE` or `GROUP BY` clause.* Failure case: *`SELECT (salary + bonus) AS total_comp FROM employees WHERE total_comp > 100000;` This breaks because the engine evaluates the `WHERE` clause (step 2)* before *the `SELECT` clause (step 5) even creates the `total_comp` alias. You must repeat the math in the `WHERE` clause.*

4.2 4.2 SELECT — Full Anatomy

The `SELECT` statement is the primary way to ask the database a question. It dictates exactly which columns to return, how to derive new values, and how to format the output.

```
SELECT [ALL | DISTINCT]
    column1, column2,                          -- explicit columns
    expression AS alias,                       -- computed column with alias
    aggregate_function(column) AS alias        -- aggregation
FROM table1
    [JOIN table2 ON condition]                 -- joins
WHERE condition                                -- row-level filter
GROUP BY column1, column2                      -- aggregation buckets
HAVING aggregate_condition                     -- group-level filter
ORDER BY column1 [ASC | DESC] [NULLS FIRST | NULLS LAST]
LIMIT n OFFSET m;                              -- pagination
```

Why it matters here: You rarely want every column from a table. Explicitly naming columns reduces network traffic and memory usage, making your queries significantly faster than using `SELECT *`.

4.3 4.3 WHERE Clause — Complete Operator Catalog

The `WHERE` clause acts as a sieve, examining every row from the source tables and discarding those that evaluate to false.

```
-- Comparison operators
WHERE salary = 50000                           -- equals
WHERE salary <> 50000                          -- not equals (also !=)
WHERE salary > 50000                           -- greater than
WHERE salary >= 50000                          -- greater than or equal
WHERE salary < 50000                           -- less than
WHERE salary <= 50000                          -- less than or equal

-- BETWEEN (inclusive range)
WHERE salary BETWEEN 40000 AND 60000
-- Equivalent to: WHERE salary >= 40000 AND salary <= 60000

-- IN (membership in a list or subquery)
WHERE dept_id IN (1, 2, 3)
```

```sql
WHERE dept_id IN (SELECT id FROM departments WHERE region = 'US')

-- LIKE (pattern matching)
WHERE name LIKE 'J%'                          -- starts with J
WHERE name LIKE '%son'                        -- ends with son
WHERE name LIKE '_a%'                         -- second character is 'a'
WHERE name LIKE '%\_test%' ESCAPE '\'         -- literal underscore
WHERE email LIKE '%@gmail.com'

-- ILIKE (case-insensitive LIKE — Postgres only)
WHERE name ILIKE 'john%'

-- SIMILAR TO (regex-like — Postgres)
WHERE name SIMILAR TO '(John|Jane)%'

-- IS NULL / IS NOT NULL
WHERE manager_id IS NULL
WHERE manager_id IS NOT NULL

-- IS DISTINCT FROM (NULL-safe equality — Postgres)
WHERE a IS DISTINCT FROM b                    -- treats NULL = NULL as FALSE
(they are not distinct)

-- ANY / ALL (compare against a set)
WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id = 3)
WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 3)

-- Logical operators
WHERE salary > 50000 AND dept_id = 3
WHERE salary > 50000 OR dept_id = 3
WHERE NOT (salary > 50000)
```

> *[!CAUTION]* ***Trap: NULL Semantics in WHERE Clauses*** *In SQL,* `NULL` *does not mean "empty string" or "zero"; it means "unknown." Because you cannot compare two unknown values,* `WHERE manager_id = NULL` *will* never *return true—it evaluates to* `NULL` *(unknown), which acts like false in a* `WHERE` *filter. You must use* `IS NULL`*.*
>
> Failure case: *Using* `NOT IN` *with a subquery that contains a* `NULL`*.* `WHERE dept_id NOT IN (1, 2, NULL)` *evaluates to unknown for every row, returning zero results. Always filter out NULLs from the subquery when using* `NOT IN`*.*

4.4 4.4 INSERT — All Variants

The `INSERT` statement adds new rows to a table. You specify the target columns and the corresponding values. If you omit a column, the database fills it with its default value (or `NULL` if no default exists).

```
-- Single row insert
INSERT INTO employees (name, salary, dept_id)
VALUES ('Alice', 75000, 3);

-- Multi-row insert
INSERT INTO employees (name, salary, dept_id)
VALUES ('Bob', 80000, 2),
       ('Charlie', 65000, 1),
       ('Diana', 90000, 3);

-- INSERT ... SELECT (copy data from another table/query)
INSERT INTO high_earners (emp_id, name, salary)
SELECT emp_id, name, salary FROM employees WHERE salary > 100000;

-- INSERT ... RETURNING (get back the inserted data — Postgres)
INSERT INTO employees (name, salary)
VALUES ('Eve', 95000)
RETURNING emp_id, name;  -- Returns the auto-generated emp_id

-- Default values
INSERT INTO employees (name) VALUES ('Frank');  -- other columns get
DEFAULTs or NULL
INSERT INTO logs DEFAULT VALUES;                -- all columns get defaults
```

> *[!TIP]* ***Heavy Concept: UPSERT (ON CONFLICT) Analogy:*** *Imagine handing a new employee roster to payroll. If the employee doesn't exist yet, you add them. If they* do *already exist, you don't create a duplicate profile; instead, you update their existing salary.*
>
> *PostgreSQL provides the `ON CONFLICT` clause to handle this gracefully in one atomic step, preventing race conditions where two identical inserts happen simultaneously.*
>
> ***Example:***
>
> ```
> INSERT INTO products (sku, name, price)
> VALUES ('ABC123', 'Widget', 9.99)
> ON CONFLICT (sku) DO UPDATE SET price = EXCLUDED.price;
> ```

> *Here, `EXCLUDED` is a special PostgreSQL table alias that holds the row we* tried *to insert. If 'ABC123' already exists (violating the unique constraint on `sku`), we catch the conflict and update the price with the proposed new value.*
>
> ***Alternative rejected:*** *Doing a `SELECT` to check for existence, followed by an `INSERT` or `UPDATE` from application code. This is a classic race condition; another process might insert the row in the millisecond between your `SELECT` and `INSERT`. `ON CONFLICT` is thread-safe and atomic.*

4.5 4.5 UPDATE — All Variants

The `UPDATE` statement modifies existing rows. It evaluates a `WHERE` clause to find target rows, then applies the `SET` expressions.

```
-- Basic update
UPDATE employees SET salary = 80000 WHERE emp_id = 42;

-- Update multiple columns
UPDATE employees
SET salary = salary * 1.10, status = 'promoted', updated_at =
CURRENT_TIMESTAMP
WHERE dept_id = 3 AND performance_rating >= 4;

-- UPDATE with a subquery
UPDATE employees
SET salary = (SELECT AVG(salary) FROM employees WHERE dept_id = 3)
WHERE emp_id = 42;

-- UPDATE ... FROM (join-based update — Postgres)
UPDATE employees e
SET salary = d.base_salary
FROM departments d
WHERE e.dept_id = d.id AND d.name = 'Engineering';

-- UPDATE ... RETURNING (Postgres)
UPDATE employees SET salary = salary * 1.10
WHERE dept_id = 3
RETURNING emp_id, name, salary;
```

> *[!CAUTION]* ***Trap: Forgetting the WHERE Clause*** *Running `UPDATE employees SET salary = 0;` without a `WHERE` clause sets* every single employee's *salary to zero. There is no "undo" button outside of a transaction. Always write the `WHERE` clause first, or wrap critical updates in a transaction with a `ROLLBACK` ready.*

4.6 4.6 DELETE — All Variants

The `DELETE` statement removes entire rows. You cannot delete a specific column (that requires an `UPDATE SET column = NULL`).

```
-- Basic delete
DELETE FROM employees WHERE emp_id = 42;

-- Delete with subquery
DELETE FROM employees
WHERE dept_id IN (SELECT id FROM departments WHERE status = 'closed');

-- Delete with USING (Postgres)
DELETE FROM employees e
USING departments d
WHERE e.dept_id = d.id AND d.status = 'closed';

-- DELETE ... RETURNING (Postgres)
DELETE FROM employees WHERE status = 'terminated'
RETURNING emp_id, name;  -- returns deleted rows for logging

-- Delete all rows (row-by-row, fully logged — compare with TRUNCATE)
DELETE FROM logs;
```

4.7 4.7 The MERGE Statement (UPSERT)

Starting in PostgreSQL 15, SQL-standard `MERGE` is supported. While `INSERT ... ON CONFLICT` is great for simple UPSERTs on a single table, `MERGE` is designed for complex data synchronization between entire tables.

> *[!TIP]* ***Heavy Concept: MERGE Analogy:*** *Reconciling a daily sales feed into a master inventory sheet. For each line in the daily feed, you check the master sheet. If the product matches, you subtract the sold quantity. If the product is entirely new in the feed, you add a new line to the master sheet.*
>
> *`MERGE` combines `INSERT`, `UPDATE`, and `DELETE` into a single unified statement based on a join condition.*
>
> ***Example:***
>
> ```
> MERGE INTO inventory AS target
> USING daily_feed AS source
> ON target.product_id = source.product_id
> WHEN MATCHED THEN
>     UPDATE SET quantity = target.quantity + source.quantity, updated_at =
> ```

```
CURRENT_TIMESTAMP
WHEN NOT MATCHED THEN
    INSERT (product_id, quantity) VALUES (source.product_id,
source.quantity);
```

Plain-English breakdown: *1.* `MERGE INTO target USING source`*: We define the table to modify (*`inventory`*) and the data to pull from (*`daily_feed`*). 2.* `ON ...`*: We specify the common identifier to match records (the* `product_id`*). 3.* `WHEN MATCHED THEN UPDATE`*: If a match is found, we update the existing row in* `inventory`*. 4.* `WHEN NOT MATCHED THEN INSERT`*: If the* `daily_feed` *has a product not present in* `inventory`*, we insert it.*

Question to sit with: *Why would you use* `MERGE` *over* `ON CONFLICT`*?* `MERGE` *can update or delete rows when they* don't *match, allowing full synchronization, whereas* `ON CONFLICT` *only reacts to unique constraint violations during insertion.*

4.8 4.8 The Relational JOIN — All Types with Examples

Relational databases store facts in isolated tables to avoid duplication (normalization). To answer real-world questions, you must stitch these tables back together.

[!TIP] ***Heavy Concept: The Relational JOIN Analogy:*** *Imagine looking up an employee in a phonebook, seeing their department ID is "3", then walking over to a separate department directory, looking up "3", and writing down "Engineering" next to the employee's name. A JOIN automates this cross-referencing.*

Example: *Assume two tables:* `employees (emp_id, name, dept_id)` *and* `departments (id, dept_name)`*.*

INNER JOIN: *Keep only rows where the lookup succeeds in both tables.*

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.id;
```

If an employee has no `dept_id`*, or a department has no employees, they are dropped from the result.*

LEFT JOIN (Outer): *Keep every row from the left table (employees). If the lookup fails, fill the right side with* `NULL`*s.*

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id;
```

Why here: *Essential for finding missing data, like "find all employees who have not been assigned a department yet."*

RIGHT JOIN (Outer): *The mirror of LEFT JOIN. Keeps every row from the right table (departments).*

```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.id;
```

FULL JOIN (Outer): *Keep everything. If the lookup succeeds, join them. If it fails on either side, fill the missing half with* `NULL`*s.*

```
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.id;
```

CROSS JOIN: *Create every possible combination (a Cartesian product).*

```
SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

100 employees × 5 departments = 500 rows. Usually useless on its own, but powerful when combined with a lateral join.

SELF JOIN: *A table joining to itself.*

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Why here: *Crucial for hierarchical data, like organizational charts or parent-child category trees.*

LATERAL JOIN (Postgres): *A loop disguised as a join.*

```
SELECT d.dept_name, top_emp.name, top_emp.salary
FROM departments d
CROSS JOIN LATERAL (
    SELECT name, salary FROM employees
```

```
    WHERE dept_id = d.id ORDER BY salary DESC LIMIT 3
) AS top_emp;
```

Normally, a subquery in a `FROM` clause cannot reference the tables joined before it. `LATERAL` breaks this rule. For every *department row, it re-runs the subquery, allowing you to fetch the "top 3 earners per department".*

[!CAUTION] ***Trap: Implicit Cartesian Products*** *Writing `SELECT * FROM A, B;` without a `WHERE` clause implicitly performs a `CROSS JOIN`. In production, this can accidentally generate billions of rows, crashing the database server through memory exhaustion. Always use explicit `JOIN ... ON ...` syntax.*

4.9 4.9 Aggregate Functions

Aggregations collapse multiple rows into a single scalar value.

```
SELECT
    COUNT(*)                AS total_rows,        -- counts all rows (incl.
NULLs)
    COUNT(salary)           AS non_null_salaries, -- counts non-NULL values
    COUNT(DISTINCT dept_id) AS unique_depts,
    SUM(salary)             AS total_payroll,
    AVG(salary)             AS avg_salary,
    MIN(salary)             AS lowest_salary,
    MAX(salary)             AS highest_salary,
    -- String aggregation (concatenate names)
    STRING_AGG(name, ', ' ORDER BY name) AS all_names,
    -- Array aggregation (Postgres)
    ARRAY_AGG(DISTINCT dept_id) AS dept_list
FROM employees;
```

The `COUNT(*)` vs `COUNT(column)` trap: `COUNT(*)` counts the number of *rows* in the table, regardless of what they contain. `COUNT(column)` specifically counts how many *non-NULL* values exist in that specific column. If an employee has a `NULL` salary, they are counted by `COUNT(*)` but ignored by `COUNT(salary)`.

4.10 4.10 GROUP BY — Advanced Grouping

When you need aggregations per category rather than across the whole table, use `GROUP BY`.

[!TIP] ***Heavy Concept: WHERE vs HAVING*** *Both filter data, but at completely different stages of execution.*

***WHERE** filters* raw rows *before they are grouped. **HAVING** filters* buckets *after they have been aggregated.*

***Example:** Find departments with an average salary over $70,000, excluding the CEO from the calculation.*

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
WHERE job_title != 'CEO'           -- Filters out the CEO (raw row)
GROUP BY dept_id                   -- Creates department buckets
HAVING AVG(salary) > 70000;        -- Filters the final buckets
```

Failure case: *`WHERE AVG(salary) > 70000;` This throws an error because at the time the `WHERE` clause runs (Step 2), the database hasn't computed any averages yet (Step 5).*

[!TIP] ***Heavy Concept: ROLLUP, CUBE, and GROUPING SETS Analogy:** Creating subtotals in a pivot table. Instead of writing three separate queries (one for detail, one for subtotals, one for the grand total) and gluing them together, you ask the database to do the math in one pass.*

GROUP BY ROLLUP (Hierarchical Subtotals):

```
SELECT dept_id, job_title, SUM(salary) AS total
FROM employees
GROUP BY ROLLUP (dept_id, job_title);
```

Produces: 1. Detail: (dept, title) 2. Subtotal: (dept, NULL) -> Total per department 3. Grand Total: (NULL, NULL) -> Total everywhere

GROUP BY CUBE (All Subtotal Combinations):

```
SELECT dept_id, job_title, SUM(salary) AS total
FROM employees
GROUP BY CUBE (dept_id, job_title);
```

Produces everything `ROLLUP` does, plus subtotals by `job_title` across all departments.

***GROUP BY GROUPING SETS (Explicit Subtotals):** When you only want specific aggregations, skipping the noise.*

```
SELECT dept_id, job_title, SUM(salary) AS total
FROM employees
```

```
GROUP BY GROUPING SETS (
    (dept_id, job_title),  -- detail
    ()                     -- grand total only (skips intermediate
subtotals)
);
```

4.11 4.11 CASE Expressions

The `CASE` expression acts as an inline `if/else` statement inside your SQL query. It returns a single value based on evaluated conditions.

```
-- Simple CASE (equality check on a single column)
SELECT name,
    CASE dept_id
        WHEN 1 THEN 'Engineering'
        WHEN 2 THEN 'Sales'
        WHEN 3 THEN 'Marketing'
        ELSE 'Other'
    END AS department_name
FROM employees;

-- Searched CASE (arbitrary boolean conditions)
SELECT name, salary,
    CASE
        WHEN salary >= 100000 THEN 'Senior'
        WHEN salary >= 60000  THEN 'Mid-Level'
        WHEN salary >= 30000  THEN 'Junior'
        ELSE 'Intern'
    END AS level
FROM employees;

-- CASE inside an aggregate (conditional aggregation)
SELECT dept_id,
    SUM(CASE WHEN salary > 70000 THEN 1 ELSE 0 END) AS high_earners,
    SUM(CASE WHEN salary <= 70000 THEN 1 ELSE 0 END) AS others
FROM employees
GROUP BY dept_id;
```

Why it matters here: Conditional aggregation (using `SUM(CASE WHEN...)`) allows you to pivot data, turning rows into columns without using complex proprietary `PIVOT` syntaxes.

4.12 4.12 NULL Handling Functions

Because `NULL` poisons calculations (e.g., `10 + NULL = NULL`), you frequently need to substitute a default value when a `NULL` is encountered.

```sql
-- COALESCE: Returns the first non-NULL argument in the list.
SELECT name, COALESCE(phone, email, 'No contact') AS contact
FROM employees;

-- NULLIF: Returns NULL if the two arguments are equal.
SELECT revenue / NULLIF(cost, 0) AS margin
FROM products;
```

Why it matters here: `NULLIF` is the standard trick to prevent division-by-zero errors. If `cost` is `0`, `NULLIF(cost, 0)` returns `NULL`. The division becomes `revenue / NULL`, which safely yields `NULL` instead of crashing the query.

4.13 4.13 Type Casting

Sometimes a value is interpreted as the wrong data type (e.g., reading a string from a CSV that should be a date). You must convert it manually.

```sql
-- CAST (Standard SQL)
SELECT CAST('2025-01-15' AS DATE);
SELECT CAST(salary AS VARCHAR(20)) FROM employees;
SELECT CAST('123.45' AS DECIMAL(10,2));

-- :: operator (Postgres shorthand, extremely common in practice)
SELECT '2025-01-15'::DATE;
SELECT salary::TEXT FROM employees;
```

4.14 4.14 Set Operations

Set operations combine the results of two or more independent `SELECT` queries into a single column structure. Both queries must output the exact same number of columns with compatible data types.

```sql
-- UNION: Append the results and remove duplicates.
SELECT name FROM employees WHERE dept_id = 1
UNION
SELECT name FROM contractors WHERE dept_id = 1;

-- UNION ALL: Append the results but keep duplicates.
SELECT name, 'employee' AS source FROM employees
UNION ALL
SELECT name, 'contractor' AS source FROM contractors;

-- INTERSECT: Only return rows that appear in BOTH query results.
SELECT email FROM employees
INTERSECT
SELECT email FROM newsletter_subscribers;
```

```
-- EXCEPT: Return rows that appear in the first query, but NOT the second.
SELECT email FROM employees
EXCEPT
SELECT email FROM unsubscribed;
```

Alternative rejected: Why use `UNION ALL` instead of `UNION`? `UNION` requires the database engine to sort the entire combined result set to find and eliminate duplicates, which is a massive performance hit. If you know the sets are already disjoint (e.g., active vs deleted users) or you don't care about duplicates, *always* use `UNION ALL`.

4.14.1 References

1. SQL Logical Execution Order - SQL Authority - https://blog.sqlauthority.com
2. PostgreSQL: Documentation: MERGE - PostgreSQL Global Development Group - https://www.postgresql.org/docs/current/sql-merge.html
3. SQL Joins - DataCamp - https://www.datacamp.com/tutorial/sql-joins
4. SQL Set Operators - Wikipedia - https://en.wikipedia.org/wiki/Set_operations_(SQL)

5 5. Advanced Querying & Syntaxes

When you move beyond basic data retrieval, you enter the territory of complex analytical queries. This section covers Subqueries, Common Table Expressions (CTEs), Window Functions, a catalog of essential built-in functions, and data pivoting.

5.1 5.1 Subqueries (Correlated vs. Non-Correlated)

A subquery is simply a query nested inside another query. The database engine handles them very differently depending on whether the inner query references data from the outer query.

> *[!TIP]* ***Heavy Concept: Correlated Subqueries***
>
> ***Analogy:*** *A* non-correlated subquery *is like asking the HR department for a printed list of valid department IDs once, and then checking your employee files against it. A* correlated subquery *is like picking up an employee file, calling HR to calculate the average salary specifically for* that employee's department*, putting the phone down, picking up the next file, and calling HR again.*
>
> ***Concrete Example:*** *Find employees who make more than their own department's average salary.*

```
SELECT name, salary, dept_id
FROM employees outer_emp
WHERE salary > (
    SELECT AVG(salary)
```

```
    FROM employees inner_emp
    WHERE inner_emp.dept_id = outer_emp.dept_id
);
```

Plain English, step by step: *1. The database starts evaluating the outer query and looks at the first employee row (e.g., Alice in Sales, `dept_id = 1`). 2. It evaluates the `WHERE` clause and encounters the subquery. 3. Because the inner query references `outer_emp.dept_id`, it is **correlated**. The engine substitutes Alice's `dept_id` into the subquery. 4. The inner query runs, computing the average salary strictly for Sales (`dept_id = 1`). 5. The outer query compares Alice's salary to this computed average. 6. The engine moves to the next employee row and **repeats the entire process**.*

Technical Statement: *A correlated subquery establishes a row-by-row dependency. The inner query cannot be evaluated independently; it must conceptually execute once for every candidate row evaluated by the outer query [1].*

The Trap: *Because of this row-by-row execution, running a correlated subquery on a table with 10 million rows means the engine might execute the inner query 10 million times. Performance will grind to a halt. When performance matters, a correlated subquery should usually be rewritten using a `JOIN` or a Window Function.*

5.1.1 Subquery Placement

Subqueries can appear in almost any clause:

```
-- Non-correlated subquery in WHERE (executes once)
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Subquery in FROM (acts as a temporary "derived table")
SELECT dept_id, avg_sal
FROM (
    SELECT dept_id, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY dept_id
) AS dept_stats
WHERE avg_sal > 70000;

-- Subquery in SELECT (must be a "scalar subquery" returning exactly 1 row
and 1 column)
SELECT name, salary,
    (SELECT AVG(salary) FROM employees) AS company_avg
FROM employees;
```

5.1.2 `EXISTS` vs `IN`

When checking for the existence of related records, `EXISTS` checks if a subquery returns *any* rows and short-circuits (stops searching) as soon as it finds one, whereas `IN` checks value membership against a fully generated list.

```
-- IN: generates the list of IDs first, then checks membership
SELECT * FROM orders
WHERE customer_id IN (SELECT id FROM customers WHERE region = 'US');

-- EXISTS: highly efficient with correlated subqueries; stops at the first
match
SELECT * FROM orders o
WHERE EXISTS (
    SELECT 1 FROM customers c
    WHERE c.id = o.customer_id AND c.region = 'US'
);
```

> *[!CAUTION]* ***Trap: `NOT IN` with `NULL` values*** *If you write `WHERE id NOT IN (SELECT parent_id FROM items)`, and even* one *`parent_id` in the subquery returns `NULL`, the entire `NOT IN` clause evaluates to `UNKNOWN` (which acts as `FALSE` for filtering). The query will mysteriously return zero rows. Always use `NOT EXISTS` when there is a risk of `NULL` values in the subquery result.*

5.2 5.2 Common Table Expressions (CTEs)

A Common Table Expression (defined via the `WITH` keyword) acts as a temporary, named result set that exists only for the duration of the query. It allows you to replace deeply nested subqueries with clean, top-to-bottom readable code.

```
WITH active_employees AS (
    SELECT * FROM employees WHERE status = 'active'
),
dept_stats AS (
    SELECT dept_id, AVG(salary) AS avg_sal
    FROM active_employees
    GROUP BY dept_id
)
SELECT ae.name, ds.avg_sal
FROM active_employees ae
JOIN dept_stats ds ON ae.dept_id = ds.dept_id;
```

> *[!TIP]* ***Heavy Concept: Recursive CTEs***

Analogy: *A recursive CTE is like a `while` loop in traditional programming that keeps adding results to a list. It starts with a base case, then continuously uses its own previous output to find the next batch of data, stopping only when a loop iteration finds absolutely nothing new.*

Concrete Example: *Building an organizational chart from a self-referencing `employees` table (where each employee has a `manager_id`).*

```sql
WITH RECURSIVE OrgChart AS (
    -- 1. The Anchor Member
    SELECT emp_id, name, manager_id, 1 AS depth
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- 2. The Recursive Member
    SELECT e.emp_id, e.name, e.manager_id, o.depth + 1
    FROM employees e
    INNER JOIN OrgChart o ON e.manager_id = o.emp_id
)
SELECT * FROM OrgChart ORDER BY depth;
```

Watch it happen step by step: *1.* ***Anchor Execution:*** *The database runs the query* before *the `UNION ALL`. This finds the CEO (no manager). Output: `[CEO]`. This is our working set. 2.* ***Recursive Iteration 1:*** *The database runs the query* after *the `UNION ALL`. Crucially, when the query references `OrgChart`, it substitutes the* working set from the previous step. *It joins the `[CEO]` against the employees table to find the CEO's direct reports. Output: `[VPs]`. This becomes the new working set, appending to the final output. 3.* ***Recursive Iteration 2:*** *It runs the recursive query again, this time substituting `[VPs]` for `OrgChart`. It finds the VP's direct reports. Output: `[Directors]`. 4.* ***Termination:*** *It repeats this until an iteration returns zero rows (e.g., when it queries for the direct reports of entry-level staff and finds none). The final result is the combined output of the anchor and all iterations [2].*

Technical Statement: *A recursive CTE requires the `RECURSIVE` modifier. It must contain a non-recursive base query (anchor) and a recursive query, combined by `UNION` or `UNION ALL`. The recursive self-reference applies strictly to the rows produced by the immediately preceding iteration.*

5.3 5.3 Window Functions

Window functions perform calculations across a set of rows related to the current row, but they fundamentally differ from standard aggregation.

[!TIP] ***Heavy Concept: The Window Function Mental Model***

> ***Analogy:*** *Standard `GROUP BY` aggregation is a meat grinder: you put in 100 rows, and 1 aggregated sausage comes out. A Window Function is a magnifying glass: you look at 100 individual rows, but the glass also shows you the average of their neighbors, without destroying the rows themselves.*
>
> ***Concrete Example:***

```
SELECT name, dept_id, salary,
    AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM employees;
```

> ***Plain English, step by step:*** *1. The query processes the `employees` table. 2. Unlike a `GROUP BY`,* ***no rows are collapsed****. Every single employee row remains intact in the output. 3. The `OVER` clause defines a "window" (a specific grouping) for the function to look at. Here, `PARTITION BY dept_id` means the window consists of all employees sharing the current row's department. 4. For each row, the `AVG()` function calculates the average salary of that specific window, and attaches the result as a new column.*
>
> ***Technical Statement:*** *A window function performs an aggregate-like calculation over a frame of rows, but retains the identity of every input row in the result set [3]. The `OVER()` clause dictates what rows the function is allowed to "see" when computing the value for the current row.*

5.3.1 Anatomy of the `OVER` Clause

The `OVER` clause can contain three sub-clauses to control the window:

1. **`PARTITION BY`**: Divides the result set into independent groups. The function's calculation resets every time the partition boundary is crossed.
2. **`ORDER BY`**: Defines the logical sequence of rows within each partition. This is mandatory for ranking functions and running totals.
3. **Frame Clause**: Defines a sliding boundary within the partition (e.g., "the 3 rows before this one").

5.3.2 1. Ranking Functions

Ranking functions assign a sequential position to rows within a partition.

```
SELECT name, dept_id, salary,
    ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
row_num,
    RANK()       OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
    DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
dense_rnk
FROM employees;
```

Tie-breaking behavior (e.g., for salaries [100k, 100k, 90k]): - `ROW_NUMBER()`: Assigns strictly sequential integers (1, 2, 3). Ties are broken arbitrarily. - `RANK()`: Assigns the same rank to ties,

but leaves a gap afterward (1, 1, 3). - `DENSE_RANK()`: Assigns the same rank to ties, with no gap afterward (1, 1, 2).

5.3.3 2. Navigation (Offset) Functions

These functions pull values from other rows relative to the current row's position.

```
SELECT name, salary, hire_date,
    -- Pull the salary from the previous row
    LAG(salary, 1) OVER (ORDER BY hire_date) AS prev_salary,
    -- Pull the salary from the next row
    LEAD(salary, 1) OVER (ORDER BY hire_date) AS next_salary,
    -- Get the first value in the window
    FIRST_VALUE(name) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
top_earner
FROM employees;
```

> *[!CAUTION]* ***Trap: LAST_VALUE default frame*** *`LAST_VALUE()` seems like it should return the absolute last row of the partition. However, if you use an `ORDER BY`, the default frame ends at the `CURRENT ROW`. Therefore, `LAST_VALUE()` will just return the current row's value! You must explicitly set the frame to `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to see the true final row.*

5.3.4 3. Aggregate Window Functions and Frames

When you pair an aggregate function (like `SUM`) with an `ORDER BY` inside the `OVER` clause, it defaults to a **running total**.

```
SELECT name, hire_date, salary,
    -- Running total (cumulative sum) of salaries over time
    SUM(salary) OVER (ORDER BY hire_date) AS cumulative_salary
FROM employees;
```

This works because adding `ORDER BY` implicitly changes the window's *frame*. - Without `ORDER BY`, the frame is the entire partition. - With `ORDER BY`, the default frame becomes `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (from the beginning of the partition up to the current row).

You can explicitly set frames for sliding calculations:

```
SELECT name, hire_date, salary,
    -- Moving average of the current row and the 2 preceding rows
    AVG(salary) OVER (
        ORDER BY hire_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
```

```
    ) AS moving_avg_3
FROM employees;
```

5.4 5.4 Built-in Functions (PostgreSQL)

PostgreSQL provides a massive standard library of data-manipulation functions [4]. Here are the core functions required for daily analysis.

5.4.1 String Functions

String functions manipulate text (`VARCHAR` / `TEXT`) data.

- **`LENGTH(string)`**: Returns the number of characters in a string.
- **`SUBSTRING(string FROM start FOR count)`**: Extracts a portion of a string. `SUBSTRING('PostgreSQL' FROM 1 FOR 4)` returns `'Post'`.
- **`POSITION(substring IN string)`**: Returns the 1-based integer index of a substring, or 0 if not found.
- **`TRIM(string)`**: Removes spaces from both the beginning and end of a string.
- **`REPLACE(string, target, replacement)`**: Replaces all occurrences of the target substring with the replacement.
- **`UPPER(string)` / `LOWER(string)`**: Converts case.
- **Concatenation (`||`)**: Joins strings together. `'Hello' || ' ' || 'World'` yields `'Hello World'`.
- **`REPEAT(string, count)`**: Repeats a string the specified number of times.

Postgres also heavily supports Regular Expressions. The `~` operator tests for a regex match (`'abc' ~ '[a-z]+'` returns true), and `REGEXP_REPLACE()` allows regex-based substitutions.

5.4.2 Date and Time Functions

Date functions handle timestamps, dates, and intervals.

- **`CURRENT_DATE` / `CURRENT_TIMESTAMP`**: Returns the current system date or timestamp. (Note: These do not use parentheses).
- **`EXTRACT(field FROM source)`**: Pulls a sub-field (like year, month, or day) from a timestamp. `EXTRACT(YEAR FROM hire_date)` returns the year as an integer.
- **`DATE_TRUNC(precision, source)`**: Truncates a timestamp down to a specific precision, zeroing out everything smaller. `DATE_TRUNC('month', CURRENT_TIMESTAMP)` returns the first day of the current month at midnight.
- **Interval Arithmetic**: You can directly add or subtract time using the `INTERVAL` keyword. `CURRENT_DATE + INTERVAL '30 days'` computes a date 30 days in the future.
- **`AGE(timestamp, timestamp)`**: Returns a human-readable interval representing the difference between two timestamps (e.g., `'2 years 3 mons'`).
- **`TO_CHAR(timestamp, format)`**: Formats a date into a specific string layout. `TO_CHAR(hire_date, 'YYYY-MM-DD')` outputs `'2025-06-27'`.
- **`TO_DATE(string, format)`**: Parses a text string back into a date object.

5.4.3 Math Functions

- **`ABS(numeric)`**: Absolute value.
- **`ROUND(numeric, decimal_places)`**: Rounds a number to the specified decimal places.
- **`TRUNC(numeric, decimal_places)`**: Truncates a number (cuts it off without rounding). `TRUNC(3.99, 0)` is 3.
- **`CEIL(numeric)`** / **`FLOOR(numeric)`**: Rounds up / rounds down to the nearest whole integer.
- **`MOD(x, y)`**: Returns the remainder of division (modulo).
- **`POWER(base, exponent)`**: Exponentiation.
- **`RANDOM()`**: Generates a random float between 0.0 and 1.0.

5.4.4 Conditional Functions

- **`COALESCE(val1, val2, ...)`**: Evaluates arguments in order and returns the **first non-null** value. Crucial for defaulting nulls: `COALESCE(bonus, 0)` returns 0 if the bonus is null.
- **`NULLIF(val1, val2)`**: Returns `NULL` if `val1` equals `val2`; otherwise returns `val1`. Useful for preventing divide-by-zero errors: `total / NULLIF(count, 0)`.
- **`GREATEST(val1, val2, ...)`** / **`LEAST(...)`**: Returns the largest or smallest value from a list of expressions.

5.5 5.5 Conditional Aggregation (Pivot / Crosstab)

A "pivot" or "crosstab" transforms row-based data into columnar data. For example, turning a column of `quarter` labels (Q1, Q2, Q3) into actual physical columns in the result set.

Unlike some other databases, PostgreSQL does not have a native `PIVOT` keyword. Instead, the standard and most flexible way to pivot data in Postgres is **Conditional Aggregation**.

This technique combines an aggregate function (like `SUM`) with conditional logic. While you can use the standard `CASE` statement, PostgreSQL provides a dedicated, elegant `FILTER` clause specifically for this purpose.

> *[!NOTE]* ***Why this works:*** *The aggregate function evaluates every row in the group, but the `FILTER` clause (or `CASE` statement) ensures that only rows matching the specific column criteria are actually fed into the sum.*

```
-- Standard SQL approach using CASE (portable)
SELECT dept_name,
    SUM(CASE WHEN quarter = 'Q1' THEN revenue ELSE 0 END) AS "Q1_Rev",
    SUM(CASE WHEN quarter = 'Q2' THEN revenue ELSE 0 END) AS "Q2_Rev",
    SUM(CASE WHEN quarter = 'Q3' THEN revenue ELSE 0 END) AS "Q3_Rev",
    SUM(CASE WHEN quarter = 'Q4' THEN revenue ELSE 0 END) AS "Q4_Rev"
FROM sales
GROUP BY dept_name;

-- Modern PostgreSQL approach using FILTER (Cleaner)
```

```sql
SELECT dept_name,
    SUM(revenue) FILTER (WHERE quarter = 'Q1') AS "Q1_Rev",
    SUM(revenue) FILTER (WHERE quarter = 'Q2') AS "Q2_Rev",
    SUM(revenue) FILTER (WHERE quarter = 'Q3') AS "Q3_Rev",
    SUM(revenue) FILTER (WHERE quarter = 'Q4') AS "Q4_Rev"
FROM sales
GROUP BY dept_name;
```

Note: PostgreSQL also offers a `crosstab()` function via the `tablefunc` extension, which can pivot dynamically without hardcoding columns, but it requires installing the extension and writing the source query as a text string. Conditional aggregation remains the standard everyday approach.

5.5.1 References

1. PostgreSQL Documentation: Subqueries - https://www.postgresql.org/docs/current/functions-subquery.html
2. PostgreSQL Documentation: WITH Queries (CTEs) - https://www.postgresql.org/docs/current/queries-with.html
3. PostgreSQL Documentation: Window Functions - https://www.postgresql.org/docs/current/tutorial-window.html
4. PostgreSQL Documentation: Functions and Operators - https://www.postgresql.org/docs/current/functions.html

6 6. Data Control & Transaction Control (DCL/TCL)

Database security and concurrency are managed through Data Control Language (DCL) and Transaction Control Language (TCL). Understanding these two languages is essential for writing applications that secure sensitive information and do not corrupt data under heavy, simultaneous load.

6.1 6.1 Data Control Language (DCL) and Roles

Data Control Language (DCL) commands manage permissions and access rights. A database is rarely accessed by a single almighty administrator. In a real system, multiple users, backend application servers, and reporting tools all connect simultaneously. DCL ensures that each connection has exactly the permissions it needs, and nothing more.

In PostgreSQL, permissions are managed via a **Role**. A role is an entity that can own database objects and have database privileges (the right to perform a specific action, like reading or inserting). A role can act as a specific user (if given a password to log in) or as a group (a collection of permissions that multiple users can inherit).

6.1.1 GRANT

The `GRANT` command issues specific privileges on a database object to a role.

```
-- Grant specific privileges on a single table
GRANT SELECT, INSERT, UPDATE ON employees TO application_role;

-- Grant highly destructive privileges to an admin role
GRANT DELETE ON employees TO admin_role;

-- Grant all available privileges on a table
GRANT ALL PRIVILEGES ON employees TO admin_role;

-- Column-level precision: Grant access to specific columns only.
-- Useful for hiding sensitive data like salaries.
GRANT SELECT (first_name, last_name, email), UPDATE (email) ON employees TO
support_role;

-- Schema-level grant: Grant read access on every existing table in a schema
(folder).
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_role;

-- Grant with the ability for the recipient to re-grant the privilege to
others.
GRANT SELECT ON employees TO lead_analyst WITH GRANT OPTION;

-- Grant execute permission to allow a role to run a stored function.
GRANT EXECUTE ON FUNCTION calculate_bonus(INT) TO hr_role;
```

6.1.2 REVOKE

The `REVOKE` command retracts privileges that were previously granted.

```
-- Revoke specific privileges
REVOKE INSERT, UPDATE ON employees FROM application_role;

-- Revoke all privileges
REVOKE ALL PRIVILEGES ON employees FROM admin_role;

-- Revoke only the ability to re-grant, keeping the underlying privilege
intact.
REVOKE GRANT OPTION FOR SELECT ON employees FROM lead_analyst;

-- Cascade revoke: If lead_analyst used their GRANT OPTION to give the
-- privilege to a junior analyst, CASCADE revokes it from the junior analyst
as well.
REVOKE SELECT ON employees FROM lead_analyst CASCADE;
```

6.1.3 Role Management and Row-Level Security

```
-- Create a user role (can log into the database)
CREATE ROLE reporting_user WITH LOGIN PASSWORD 'secure_pass_123';

-- Create a group role (cannot log in directly, acts as a permissions
container)
CREATE ROLE analyst_group NOLOGIN;

-- Assign the group role to the user, granting them all analyst privileges
GRANT analyst_group TO reporting_user;

-- Drop (delete) a role entirely
DROP ROLE IF EXISTS analyst_group;
```

PostgreSQL also supports **Row-Level Security (RLS)**. While standard `GRANT` commands filter which *tables* or *columns* a user can see, RLS filters which *rows* a user can see.

```
-- Enable RLS on the table
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;

-- Create a policy so users can only see rows belonging to their own
department
CREATE POLICY emp_department_policy ON employees
    FOR SELECT
    USING (dept_id = current_setting('app.current_dept')::INT);
```

6.2 6.2 Transaction Control & ACID Properties

A transaction is a logical unit of work that contains one or more SQL statements. A database must guarantee that a transaction executes completely and safely, even if a power cord is pulled mid-execution, or if two servers try to modify the same data at the exact same millisecond.

To maintain data integrity, relational databases guarantee the **ACID** properties:

- **Atomicity:** The "all-or-nothing" rule. If a transaction contains five `UPDATE` statements, and the fifth one fails, the database automatically undoes the first four. A transaction cannot partially complete.
- **Consistency:** The transaction must transition the database from one valid state to another valid state. All constraints (like `NOT NULL` or `CHECK`) must be honored. If a transaction attempts to violate a constraint, it is rejected entirely.
- **Isolation:** Concurrent transactions execute as if they are the only transaction running on the system. They must not interfere with each other's intermediate, uncommitted states. (We will explore this heavily in Section 6.3).
- **Durability:** Once the database engine confirms a transaction is committed, the changes are permanent. They are written to non-volatile storage and will survive a hard crash or power failure.

6.2.1 TCL Commands (Transaction Control Language)

You manage transactions explicitly using TCL commands.

```
-- Begin a new transaction block
BEGIN;

-- Commit the transaction, permanently saving all changes to disk
COMMIT;

-- Rollback the transaction, instantly undoing all changes made since BEGIN
ROLLBACK;
```

You can also create a **Savepoint** within a transaction. This allows you to partially roll back to a specific checkpoint without abandoning the entire transaction.

```
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;

-- Create a checkpoint
SAVEPOINT before_risky_update;
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;

-- If the second update hits a constraint error (e.g., balance exceeded
limit):
ROLLBACK TO SAVEPOINT before_risky_update;

-- The first update (id = 1) is preserved. We can now try something else or
commit.
COMMIT;
```

6.2.2 Practical Transaction Example: A Bank Transfer

A classic example of atomicity is transferring money. It fundamentally requires two steps: subtracting from Account A, and adding to Account B. If the database crashes between the two steps, money vanishes. A transaction prevents this.

```
BEGIN;

-- Step 1: Debit from account A
UPDATE accounts SET balance = balance - 500.00
WHERE account_id = 'A' AND balance >= 500.00;

-- Step 2: Credit to account B
UPDATE accounts SET balance = balance + 500.00
WHERE account_id = 'B';
```

```
-- Step 3: Permanently apply both changes
COMMIT;
-- If the server loses power during Step 2, the database restarts,
-- sees an uncommitted transaction, and completely reverses Step 1.
```

6.3 6.3 The Isolation Trap: Read Phenomena

Concurrency is fundamentally unintuitive. If 100 users try to read and write the exact same rows simultaneously, the database must use locks to prevent chaos. However, locking every row for every query halts performance to a crawl. You can tune the trade-off between strict accuracy and high performance using **Isolation Levels**. To tune them safely, you must understand the bugs—known as read phenomena—that loose isolation levels allow.

> *[!TIP]* ***Heavy Concept: Read Phenomena & Isolation Levels Analogy:*** *Imagine a shared Google Doc being edited by your coworker. -* ***Dirty Read:*** *You read a paragraph your coworker just typed. Based on that paragraph, you make a decision. But your coworker hasn't hit 'Save' yet, and they suddenly hit* `Ctrl+Z` *to erase it. You just acted on information that technically never existed in the permanent record. -* ***Non-Repeatable Read:*** *You read the title of the document. You look away to check your notes. You look back, and the title is entirely different because your coworker just saved a change. Your read cannot be repeated. -* ***Phantom Read:*** *You count the number of bullet points in a list (there are 5). You look away. You look back and recount, and now there are 6, because a coworker just inserted a brand new bullet point.*
>
> ***Concrete Example (Watch a Non-Repeatable Read happen):*** *You are building an accounting app to sum a user's net worth. The user has a Checking account ($500) and a Savings account ($500). Total: $1000.* Time 1: *Your app (Transaction 1) starts querying. It reads Checking: $500.* Time 2: *Before your app reads Savings, a scheduled bill payment (Transaction 2) fires. It transfers $100 from Checking to Savings. Checking is now $400, Savings is $600. Transaction 2 commits.* Time 3: *Your app (Transaction 1) resumes and reads Savings. It sees the new, committed value: $600.* Time 4: *Your app sums the values it read: $500 (Checking, read at Time 1) + $600 (Savings, read at Time 3) =* ***$1100****.* Failure case: *Money was double-counted. The database state was entirely valid, but because the isolation level allowed a row to change* between *your reads, your transaction calculated an impossible reality. This breaks when absolute point-in-time consistency across multiple rows is required.*
>
> ***Plain English:*** *The SQL standard defines four Isolation Levels. As you move up the ladder, the database engine enforces stricter locking, preventing more phenomena, but severely slowing down simultaneous users.*
>
> ***Technical (The Four Levels):*** *1.* ***Read Uncommitted:*** *The database does zero locking. Allows Dirty Reads, Non-Repeatable Reads, and Phantom Reads. Extremely fast, but offers virtually no data integrity. (PostgreSQL safely treats this identically to* `Read Committed`*). 2.* ***Read Committed:*** *(The default in PostgreSQL). Prevents Dirty Reads. You only read committed data. However, as the bank example showed, if a separate transaction modifies a row while your transaction is*

> *running, a second read of that row yields a different result (Non-Repeatable Read). 3.* ***Repeatable Read:*** *Prevents Dirty and Non-Repeatable reads. If you read a row, the database engine guarantees that if you read it again in the same transaction, it will be identical. It achieves this by taking a snapshot. However, standard SQL says a separate transaction could still* `INSERT` *a brand new row that matches your query conditions (Phantom Read). 4.* ***Serializable:*** *The strictest level. The database guarantees that if transactions run concurrently, the result will be identical to as if they were forced to run sequentially, one-by-one in a queue. Prevents all phenomena, but causes massive lock contention, causing transactions to wait or fail entirely.*

6.3.1 Isolation Level Summary Table

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
Read Uncommitted	Possible	Possible	Possible	Fastest
Read Committed (Default)	Prevented	Possible	Possible	Fast
Repeatable Read	Prevented	Prevented	Possible (Prevented in Postgres*)	Moderate
Serializable	Prevented	Prevented	Prevented	Slowest

**Note: PostgreSQL's implementation of Repeatable Read is highly advanced and actually prevents Phantom Reads as well, though the SQL standard allows them.*

6.3.2 Explicit Row Locking (`SELECT ... FOR UPDATE`)

Instead of changing the global isolation level to Serializable, which slows down the entire system, you can use explicit row-level locking. By adding `FOR UPDATE` to a `SELECT` query, you command the database to lock the specific rows you just read, preventing any other transaction from modifying them until your transaction commits.

```
BEGIN;

-- Lock the specific row for Account 'A'.
-- Any other transaction trying to modify Account 'A', or lock it, will
PAUSE and WAIT.
SELECT balance FROM accounts WHERE account_id = 'A' FOR UPDATE;

-- Safe to perform calculations here without fear of another transaction
interfering
```

```sql
UPDATE accounts SET balance = balance - 500 WHERE account_id = 'A';

COMMIT;
-- The lock is released upon COMMIT. Waiting transactions can now proceed.
```

Advanced locking options for high-concurrency systems:

```sql
-- FOR UPDATE NOWAIT: Instead of waiting indefinitely for another
transaction
-- to release its lock, the database immediately throws an error so your app
can retry.
SELECT * FROM accounts WHERE account_id = 'A' FOR UPDATE NOWAIT;

-- FOR UPDATE SKIP LOCKED: Extremely useful for job queues.
-- It queries for rows, but simply skips over any rows that are currently
locked
-- by other workers, ensuring no two workers process the same job, without
waiting.
SELECT * FROM background_tasks
WHERE status = 'pending'
ORDER BY created_at LIMIT 1
FOR UPDATE SKIP LOCKED;
```

6.3.3 References

1. Transaction Isolation Levels - PostgreSQL Docs - https://www.postgresql.org/docs/current/transaction-iso.html
2. Row-Level Security - PostgreSQL Docs - https://www.postgresql.org/docs/current/ddl-rowsecurity.html
3. Explicit Locking - PostgreSQL Docs - https://www.postgresql.org/docs/current/explicit-locking.html

7 7. Query Optimization & Execution Under the Hood

When you write a SQL query, you are describing *what* data you want, not *how* to get it. The database engine's job is to figure out the "how." Understanding this lifecycle separates developers who guess at performance from those who engineer it.

7.1 7.1 The Query Lifecycle

Every SQL query passes through three main phases before returning data:

1. **Parser:** Checks the syntax and semantics. Are the keywords correct? Do the tables and columns actually exist? Does the user have permission to see them?
2. **Optimizer:** The brain of the database. It generates multiple possible "Execution Plans" (strategies for retrieving the data) and picks the one it estimates will be fastest.
3. **Executor:** The engine that runs the chosen execution plan. It interacts with the storage engine to fetch physical blocks of data from memory or disk, following the exact steps the optimizer laid out.

7.2 7.2 Reading Execution Plans

To see the optimizer's chosen plan in PostgreSQL, you use the `EXPLAIN` command.

```
-- Shows the estimated plan WITHOUT executing the query.
EXPLAIN SELECT * FROM employees WHERE dept_id = 3;

-- Actually EXECUTES the query and shows real timing + row counts.
EXPLAIN ANALYZE SELECT * FROM employees WHERE dept_id = 3;

-- Maximum detail: includes buffer (memory/disk) hits and outputs as JSON.
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT * FROM employees WHERE
dept_id = 3;
```

> *[!TIP]* ***Heavy Concept: Execution Plan Operators Analogy:*** *Imagine being told to find every employee in the marketing department and match them with their office floor. You could walk to every single desk in the company and ask (Table Scan), or you could look up "Marketing" in the company directory and only visit those desks (Index Scan). Once you have the names, you could look up each person one by one in the floor directory (Nested Loop), or you could memorize the floor directory first and just check your memory (Hash Join).*
>
> ***Plain English:*** *An execution plan is a tree of "operators." Data flows from the bottom (leaf nodes) up to the top. Each operator represents a specific physical algorithm the executor uses to retrieve or combine data.*

Scan Operators (How data is read from tables/indexes): - **Seq Scan (Sequential Scan):** The executor physically reads every single page of the table from disk to memory, checking every row against the `WHERE` clause. This is necessary if you need most of the table, but devastatingly slow if you only need a few rows from a massive table. - **Index Scan:** The executor traverses a B-Tree index structure to find the exact locations of the requested rows, then visits the main table (the heap) to retrieve the rest of the row data. - **Index Only Scan:** The executor finds the data it needs purely within the index structure and does not need to visit the main table at all. This is the fastest possible retrieval method, provided the index "covers" all columns requested in the `SELECT` clause. - **Bitmap Index Scan & Bitmap Heap Scan:** When reading many rows from an index, visiting the table for each row individually causes random I/O. Instead, Postgres first scans the index to build a "bitmap" (a list of memory blocks where the matching rows live)

via a *Bitmap Index Scan*. Then, a *Bitmap Heap Scan* visits those physical blocks sequentially. This is highly efficient for range queries or when combining multiple indexes (e.g., `WHERE status = 'active' OR dept_id = 3`).

Join Operators (How two datasets are combined): - **Nested Loop:** For every single row in the first (outer) dataset, the executor scans the second (inner) dataset to find matches. Excellent when the outer set is very small and the inner set has an index on the join column. - **Hash Join:** The executor takes the smaller of the two datasets and builds an in-memory hash table keyed by the join column. It then scans the larger dataset one row at a time, probing the hash table for matches. The go-to method for joining large, unsorted datasets. - **Merge Join:** If both datasets are already sorted by the join column (perhaps because they were read via an Index Scan), the executor steps through both sets simultaneously, merging them together. Excellent for massive datasets that are already ordered.

7.3 7.3 The Cost-Based Optimizer (CBO)

Historically, databases used Rule-Based Optimizers (e.g., "if an index exists, always use it"). Today, PostgreSQL and all major RDBMS use a **Cost-Based Optimizer (CBO)**.

> *[!TIP]* ***Heavy Concept: Cost-Based Optimizer & Statistics Analogy:*** *You need to drive from New York to Boston. A Rule-Based GPS says "Always take the highway because highways are faster." A Cost-Based GPS checks the live traffic data (statistics) and says "The highway is jammed with 10 miles of stopped cars; today, the backroads will cost 30 fewer minutes."*
>
> ***Example:*** *You query* `SELECT * FROM users WHERE status = 'active'`*. If 99% of users are 'active', using an index requires reading the index* and *the table, which is slower than just reading the table directly. The CBO calculates this and chooses a Seq Scan. If only 1% are 'active', the CBO calculates that an Index Scan is cheaper.*
>
> ***Plain English:*** *The CBO calculates a mathematical "cost" for every possible execution plan by estimating CPU cycles, memory usage, and disk I/O. To make accurate estimates, it relies entirely on* ***Statistics****—metadata about your tables. Statistics include the total row count, the number of distinct values in each column, and data distribution histograms.*
>
> ***The Trap: Stale Statistics*** *A common production disaster is a query that ran in 1 second yesterday suddenly taking 5 minutes today, even though the query didn't change.* Failure case: *This breaks when you do a massive data load (e.g., inserting 10 million rows) but the database statistics have not been updated. The CBO still thinks the table has 1,000 rows. It chooses a Nested Loop join (great for small tables) instead of a Hash Join, resulting in billions of CPU cycles.*
>
> ***Question to sit with:*** *If a table is extremely small (e.g., 50 rows), will the CBO ever choose an Index Scan over a Sequential Scan? Why or why not?*

In PostgreSQL, the `ANALYZE` command updates these statistics. The autovacuum daemon usually runs this automatically in the background, but manual runs are required immediately after bulk data changes.

```
-- Update statistics for a specific table
ANALYZE employees;

-- Update statistics for the entire database
ANALYZE;
```

7.4 7.4 SARGable Queries (The Index Killer)

When tuning a slow query, developers often add an index, only to find the query is *still* slow. This is usually because the query is not **SARGable**.

> *[!TIP]* ***Heavy Concept: SARGability (Search ARGument-ABLE) Analogy:*** *Looking for someone named "Smith" in a phone book. If I ask "Find Smith," you jump straight to the 'S' section and use the alphabetized nature of the book to find the exact page. If I ask "Find everyone whose last name has exactly 5 letters," the alphabetical order doesn't help you at all. You have to read every single name in the book from page 1 to 1000, counting the letters.*
>
> ***Example:*** *Consider* `WHERE UPPER(name) = 'JOHN'`*. The index on* `name` *stores names in their original case ('John', 'jOhN', 'JOHN'). The database cannot know which of those entries will equal 'JOHN' after the* `UPPER()` *function is applied without actually applying the function to every row in the index.*
>
> ***Plain English:*** *A query is SARGable if the database engine can effectively use an index to jump straight to the data. If you wrap an indexed column in a function, perform arithmetic on it, or use a leading wildcard, the database must evaluate that expression for every single row, rendering the index useless and forcing a Seq Scan.*
>
> ***Technical:*** *To utilize a B-Tree index, the search argument must be an operator that matches the index's sort order (e.g., =, >, <).*
>
> ***Alternative rejected:*** *You could fetch all rows into your application memory (e.g., in Python or Java) and filter the data there using flexible code. But this sacrifices database-level indexing entirely, incurs massive network transfer costs, and shifts the CPU burden to the app server.*

7.4.1 SARGable vs Non-SARGable — Examples

Non-SARGable (Bad)	SARGable (Good)	Why
`WHERE EXTRACT(YEAR FROM order_date) = 2025`	`WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01'`	Function on column hides the value
`WHERE salary + 1000 > 50000`	`WHERE salary > 49000`	Arithmetic on column hides the value
`WHERE LEFT(name, 3) = 'Joh'`	`WHERE name LIKE 'Joh%'`	Function on column
`WHERE name LIKE '%son'`	Cannot be SARGable (leading wildcard)	Alphabetical sort is useless if the first letter is unknown
`WHERE id::TEXT = '42'`	`WHERE id = 42`	Type casting forces evaluation

In PostgreSQL, you can fix some non-SARGable queries by creating an **Expression Index** that pre-computes the function:

```
-- The index stores the uppercase version of the name
CREATE INDEX idx_upper_name ON employees (UPPER(name));

-- Now this query IS SARGable, because it matches the indexed expression
exactly:
SELECT * FROM employees WHERE UPPER(name) = 'JOHN';
```

7.5 7.5 Common Performance Anti-Patterns

```
-- Anti-pattern 1: SELECT * (fetches all columns)
-- This defeats Index Only Scans because the index likely doesn't cover
every column.
SELECT * FROM employees;              -- Bad
SELECT name, salary FROM employees;   -- Good: only fetch what you need

-- Anti-pattern 2: Correlated subquery (runs inner query per row)
-- For every employee, the database executes the orders count query. (N+1
queries)
SELECT name, (SELECT COUNT(*) FROM orders WHERE orders.emp_id = e.id)
FROM employees e;                     -- Bad
-- Rewrite as a JOIN so the optimizer can use Hash or Merge joins:
SELECT e.name, COUNT(o.id) FROM employees e
LEFT JOIN orders o ON o.emp_id = e.id GROUP BY e.name;
```

```
-- Anti-pattern 3: DISTINCT to hide a bad JOIN
-- Masks duplicate rows resulting from an incorrect one-to-many join
relationship.
SELECT DISTINCT e.name FROM employees e
JOIN orders o ON o.emp_id = e.id;    -- Bad
-- Fix the JOIN logic or use an EXISTS clause instead of using DISTINCT as a
band-aid.

-- Anti-pattern 4: ORDER BY + LIMIT without an index
-- Without an index on created_at, the database must sort ALL rows in
memory, then return 10.
SELECT * FROM logs ORDER BY created_at DESC LIMIT 10;
-- Fix: CREATE INDEX idx_logs_created ON logs (created_at DESC);
```

7.5.1 References

1. Using EXPLAIN - PostgreSQL Docs - https://www.postgresql.org/docs/current/using-explain.html
2. Planner Statistics - PostgreSQL Docs - https://www.postgresql.org/docs/current/planner-stats.html
3. SARGable queries - SQL Authority - https://blog.sqlauthority.com/
4. Index Tuning - Brent Ozar - https://www.brentozar.com/

8 8. Indexing Strategies (B-Tree, Hash, etc.)

Adding an index to a database is the primary way developers fix slow queries. However, blindly adding indexes without understanding their underlying data structures and physical storage implications leads to bloated, slow-writing databases.

8.1 8.1 Indexing Basics

An index is a redundant data structure that trades storage space and write performance (slower `INSERT`, `UPDATE`, `DELETE`) for drastically improved read performance (faster `SELECT`).

Without an index, the database must perform a **Sequential Scan**: reading every single physical block of the table from disk to see if the rows inside match the `WHERE` clause. With an index, the database performs an **Index Scan**: navigating a specialized data structure to jump directly to the exact locations of the relevant rows on disk, bypassing the rest of the table.

8.2 8.2 Index Types — The Physical Mechanisms

PostgreSQL provides several different index types, each using a fundamentally different physical data structure optimized for specific kinds of searches.

8.2.1 B-Tree Index (The Default)

If you type `CREATE INDEX`, PostgreSQL creates a B-Tree (Balanced Tree).

> *[!TIP]* ***Heavy Concept: The B-Tree Mechanism Analogy:*** *A B-Tree is like a massive, rigorously alphabetized phone book with an intelligent table of contents. If you are looking for "Smith," you don't read every page. You check the table of contents to find that 'S' starts on page 500, then you jump there. You can easily find an exact match ("Smith"), or a range ("Everyone from Smith to Stevens").*
>
> ***Example:*** *You index a `last_name` column. The database pulls out every last name, sorts them alphabetically, and builds a tree structure. The "root" node points to "Names A-M" and "Names N-Z". The "leaf" nodes at the bottom contain the actual names in order, alongside a pointer to where that physical row lives in the main table.*
>
> ***Plain English:*** *A B-Tree is a self-balancing tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in logarithmic time. Because the data is physically sorted within the index, B-Trees perfectly support equality (=), ranges (>, <), and pattern matching that starts with a known character (`LIKE 'Joh%'`).*
>
> ***Question to sit with:*** *If a B-Tree keeps data sorted to support ranges, why is a query like `WHERE name LIKE '%Smith'` (leading wildcard) unable to use the B-Tree efficiently?*

```
CREATE INDEX idx_name ON employees (last_name);
```

8.2.2 Hash Index

> *[!TIP]* ***Heavy Concept: The Hash Mechanism Analogy:*** *A Hash index is like a coat check. You hand the attendant tag #402, and they hand you your coat instantly. There is no alphabetical order. If you ask the attendant for "all coats between 400 and 450," they cannot help you without checking every single coat, because the coats are not hung sequentially.*
>
> ***Example:*** *You create a hash index on an `email` column. The database runs every email string through a mathematical hashing algorithm that outputs a fixed integer (e.g., `john@example.com` becomes `849271`). It stores that integer in a hash table pointing to the row.*
>
> ***Plain English:*** *Hash indexes use a hash function to map keys to specific storage "buckets". They only support exact equality matches.*
>
> ***Technical:*** *Because they do not store data in sorted order, they cannot answer range queries (>) or sorting (`ORDER BY`). They are slightly faster and smaller than B-Trees for pure equality lookups.*

```
-- Best for equality-only lookups on large text fields
CREATE INDEX idx_email_hash ON employees USING HASH (email);
```

8.2.3 GIN Index (Generalized Inverted Index)

A standard B-Tree indexes a single value per row. But what if a column contains multiple values, like an array or a JSON document?

> *[!TIP]* ***Heavy Concept: The Inverted Index Mechanism Analogy:*** *Think of the index at the very back of a textbook. The textbook has many pages (rows). Each page contains many words (array elements). The inverted index lists every unique word in alphabetical order, and next to each word, it lists every page number where that word appears.*
>
> ***Plain English:*** *A GIN index takes a composite data type (like a JSON object, an array, or a full-text document), breaks it apart into its individual elements, and creates a separate index entry for every single element. It maps from the element back to the row, rather than from the row to the element.*
>
> ***Failure case:*** *GIN indexes are expensive to update. Inserting one row (e.g., a document with 500 unique words) requires inserting 500 separate entries into the index, heavily slowing down write operations.*

```
-- Indexing a JSONB column
CREATE INDEX idx_meta ON events USING GIN (metadata);
-- Supports searching for a specific key/value deep inside the JSON:
SELECT * FROM events WHERE metadata @> '{"type":"click"}';

-- Full-text search in PostgreSQL
ALTER TABLE articles ADD COLUMN search_vector TSVECTOR;
UPDATE articles SET search_vector = to_tsvector('english', title || ' ' ||
body);
CREATE INDEX idx_fts ON articles USING GIN (search_vector);

SELECT * FROM articles WHERE search_vector @@ to_tsquery('english',
'database & sql');
```

8.2.4 GiST (Generalized Search Tree) and BRIN

- **GiST:** Used for data that overlaps or has geometric properties (e.g., "Is this point inside this polygon?", "Do these two time ranges overlap?"). A GiST index organizes data into overlapping bounding boxes rather than a strict sorted list.
- **BRIN (Block Range Index):** Used for massive, naturally ordered tables (like append-only time-series logs). Instead of indexing every row, BRIN stores the minimum and maximum values for physical *blocks* of rows. It is incredibly small. If you query for events on Tuesday, it checks the block summaries, realizes a block only contains Monday data, and skips the entire block.

```
-- BRIN on a naturally ordered time-series table
CREATE INDEX idx_logs_time ON logs USING BRIN (created_at);
```

8.3 8.3 Physical Storage: Postgres Heap vs. Clustered Indexes

To understand indexing fully, you must understand how the actual table data is stored on disk.

> *[!TIP]* ***Heavy Concept: The Heap and physical ordering Analogy:*** *Imagine writing a logbook. As events happen, you simply write them on the next available blank line at the bottom of the page. This is a Heap. To find things faster, you create a separate alphabetized index card box (the B-Tree) that tells you which page and line number a specific event is written on.*
>
> ***Plain English:*** *In PostgreSQL, tables are stored as a* ***Heap****. A heap is an unordered collection of data pages. When you* `INSERT` *a row, Postgres places it in the first available free space. The B-Tree indexes you create are entirely separate data structures containing pointers (Block and Tuple IDs) back to the unordered heap.*

The "Clustered Index" Concept In other database systems (like SQL Server or MySQL's InnoDB), the primary key is a true *Clustered Index*. This means the table itself *is* a B-Tree. The actual row data is stored in the leaf nodes of the index, physically forcing the data to exist in sorted order on disk.

PostgreSQL does **not** use clustered indexes as its primary storage mechanism; it strictly uses the Heap + separate index model. However, Postgres provides a one-time command to mimic the performance benefits of clustered storage:

```
-- Physically reorder the heap on disk to match the index order
CLUSTER orders USING idx_order_date;
```

> *[!CAUTION]* ***The Trap: CLUSTER is not maintained*** *Running* `CLUSTER` *in PostgreSQL rewrites the table so the physical rows match the index order, drastically speeding up range queries because matching rows are packed into the same physical disk blocks. However, this is a* ***one-time operation****. Subsequent* `INSERT`*s and* `UPDATE`*s will go into the heap's free space, not into sorted order. You must run* `CLUSTER` *periodically (which locks the table) to maintain the physical sort.*
>
> ***Question to sit with:*** *If* `CLUSTER` *is a one-time operation, what happens to the physical order of the heap if you delete 10% of the rows and then insert new ones over time?*

8.4 8.4 Advanced Index Strategies

8.4.1 Composite Index & The Leftmost Prefix Rule

You can index multiple columns together. The order of those columns fundamentally dictates how the index can be used.

> *[!TIP]* ***Heavy Concept: Leftmost Prefix Rule Analogy:*** *Think of a telephone book sorted first by Last Name, then by First Name. If you want to find "John Smith," you find the "Smith" section,*

> *and within that, the "John" entries. The book is perfectly organized for this. But what if you want to find everyone whose First Name is "John"? The telephone book is useless. "John" appears scattered under Adams, under Baker, under Smith, and under Zimmer. Because the book is sorted by Last Name first, you cannot perform a targeted search using* only *the First Name.*
>
> ***Plain English:*** *A composite B-Tree sorts its entries strictly by the first column, then resolves ties using the second column, and so on. If a query's `WHERE` clause does not include the first (leftmost) column of the index, the database has no sorted structure to traverse, and it cannot use the index.*
>
> ***Alternative rejected:*** *You could create a separate, individual index for every single column in the table to cover all possible query combinations. This is rejected because every index slows down `INSERT`/`UPDATE` operations and consumes memory; composite indexes solve multi-column queries with fewer overall data structures.*

```
CREATE INDEX idx_composite ON orders (customer_id, order_date, status);

-- The "leftmost prefix" rule in action:
-- USES INDEX:
SELECT * FROM orders WHERE customer_id = 42;
-- USES INDEX:
SELECT * FROM orders WHERE customer_id = 42 AND order_date > '2025-01-01';
-- CANNOT USE INDEX (skipped the leftmost column):
SELECT * FROM orders WHERE order_date > '2025-01-01';
```

8.4.2 Covering Index (INCLUDE)

When an Index Scan finds a matching entry, it must normally jump to the main table (the heap) to retrieve the rest of the row's data. If you only need a few columns, you can append them to the index structure itself.

```
-- 'INCLUDE' adds the column to the leaf nodes of the index without sorting
by it
CREATE INDEX idx_orders_cover ON orders (customer_id) INCLUDE (order_date,
total);

-- Because customer_id is the key, and order_date/total are included in the
index,
-- the database performs an "Index Only Scan". It never visits the heap.
SELECT order_date, total FROM orders WHERE customer_id = 42;
```

8.4.3 Partial Indexes

If you have a massive `users` table but only ever query for active users, indexing everyone wastes disk space and memory. A partial index only indexes rows that meet a condition.

```
-- Creates a tiny index containing only active users
CREATE INDEX idx_active_users ON users (email) WHERE status = 'active';
```

8.5 8.5 When NOT to Index

- **Low Cardinality Columns:** Indexing a `gender` or `is_active` boolean column is rarely useful. If looking up 'active' returns 50% of the table, jumping back and forth between the index and the heap is slower than just sequentially scanning the heap.
- **Tiny Tables:** If a table has a few hundred rows, it fits in a single physical disk page. Sequential scans are instantaneous.
- **Heavy Write Tables:** Every `INSERT`, `UPDATE`, or `DELETE` requires updating every index on that table. Over-indexing murders write performance.

8.5.1 References

1. Index Types - PostgreSQL Docs - https://www.postgresql.org/docs/current/indexes-types.html
2. Multicolumn Indexes - PostgreSQL Docs - https://www.postgresql.org/docs/current/indexes-multicolumn.html
3. Index-Only Scans and Covering Indexes - PostgreSQL Docs - https://www.postgresql.org/docs/current/indexes-index-only-scans.html
4. CLUSTER - PostgreSQL Docs - https://www.postgresql.org/docs/current/sql-cluster.html
5. SQL Indexing and Tuning e-Book - Use The Index, Luke - https://use-the-index-luke.com/

9 9. Database Administration Basics

Even as a software engineer, you will be expected to understand the operational side of databases. Interviewers ask these questions to see if you understand how to keep data safe, highly available, and secure at scale.

9.1 9.1 Backups and Recovery

A backup strategy is dictated by two business metrics: **RPO** (Recovery Point Objective — how much data can we lose?) and **RTO** (Recovery Time Objective — how fast must we be back online?).

> *[!TIP]* ***Heavy Concept: Point-in-Time Recovery (PITR) Analogy:*** *Imagine writing a book. A* ***full backup*** *is photocopying the entire manuscript every midnight. If you spill coffee on it at 4:00 PM, you lose 16 hours of work. But what if you set up a camera that records every single keystroke as you type? That is the* ***Write-Ahead Log (WAL)****.*
>
> ***Concrete Example:*** *At 2:15 PM, a developer accidentally drops the* `users` *table. You don't restore last night's full backup and accept the data loss. Instead, you restore midnight's full backup into a fresh server, and then "play back" the WAL keystroke-by-keystroke, stopping exactly at 2:14:59 PM.*

> ***Plain English:*** *PostgreSQL (like all major relational databases) writes every change to a sequential log* before *writing it to the actual data files. By combining a periodic snapshot (a base backup) with an unbroken archive of these logs, you can reconstruct the database state at any arbitrary microsecond.*
>
> ***Technical:*** *In PostgreSQL, you take a base backup using* `pg_basebackup`*. As the database runs, it archives its WAL segments. To recover, you configure recovery settings with* `recovery_target_time = '2026-06-30 14:14:59'`*, and PostgreSQL will automatically replay transactions up to that exact timestamp and halt.*
>
> ***The Trap:*** *Thinking a daily* `pg_dump` *is a sufficient backup strategy for a production application.* `pg_dump` *produces a logical backup (SQL statements), which is slow to restore and does not support PITR. For true disaster recovery, you need physical backups (*`pg_basebackup`*) combined with WAL archiving.*

9.2 9.2 Scaling the Database (Replication & Sharding)

When a single database server maxes out its CPU or disk I/O, you must scale it.

> *[!TIP]* ***Heavy Concept: Replication vs Sharding Analogy:*** *Your pizza restaurant is overwhelmed with orders. -* ***Replication*** *is hiring an assistant who perfectly copies your recipe book and handles all the customers asking "What are the ingredients?" (Reads), leaving you to handle the actual cooking (Writes). -* ***Sharding*** *is opening a second restaurant across town. You take half the menu, they take the other half. You both handle Reads and Writes, but only for your specific pizzas.*
>
> ***Concrete Example (Replication):*** *Your analytics team runs massive* `SELECT` *queries that slow down the database for regular users. You set up a Replica. The analytics team points their queries at the Replica, while the main application continues writing to the Primary.*
>
> ***Plain English - Replication:*** *One server (Primary) accepts all writes. It continuously streams its transaction logs to one or more Read Replicas. This scales* Read *performance and provides failover redundancy. PostgreSQL supports* ***Streaming Replication*** *(copying raw bytes) and* ***Logical Replication*** *(copying specific tables or changes). Replication can be* ***Asynchronous*** *(Primary acknowledges the write immediately, Replica catches up later) or* ***Synchronous*** *(Primary waits for the Replica to confirm it saved the data, slowing down writes but guaranteeing zero data loss if the Primary dies).*
>
> ***Plain English - Sharding:*** *Partitioning the actual data horizontally across multiple servers (e.g., Server A holds users A-M, Server B holds users N-Z). This scales* Write *performance because no single server handles all inserts.*
>
> ***The Trap:*** *Sharding a relational database manually. If you shard, queries that cross shards (like joining a user on Server A to an order on Server B) become incredibly slow and complex because the database must fetch data over the network.*

9.3 9.3 Database Security

Security is implemented in layers, adhering to the Principle of Least Privilege.

- **Connection Security (`pg_hba.conf`):** PostgreSQL controls exactly who can connect, from which IP addresses, and using which authentication method via the Host-Based Authentication file.
- **Role-Based Access Control (RBAC):** Users should never connect to the database as the `postgres` superuser. The application should connect using a role (a PostgreSQL concept that encompasses both users and groups) that only has `SELECT`, `INSERT`, `UPDATE` permissions on specific tables, with no `DROP` privileges.
- **Row-Level Security (RLS):** A powerful PostgreSQL feature where you can restrict which *rows* a role can see. For example, `CREATE POLICY user_policy ON accounts USING (tenant_id = current_setting('app.current_tenant'));` ensures a user can only ever select rows belonging to their tenant, even if they run `SELECT * FROM accounts`.
- **Encryption at Rest:** Protecting the physical files on the hard drive. If a hacker steals the hard drive, they cannot read the files. (Note: PostgreSQL relies on filesystem-level encryption like LUKS or third-party extensions for transparent data encryption).
- **Encryption in Transit:** Protecting the data as it travels over the network from the database server to the application server using SSL/TLS.

9.3.1 References

1. Continuous Archiving and Point-in-Time Recovery (PITR) - PostgreSQL Documentation - https://www.postgresql.org/docs/current/continuous-archiving.html
2. High Availability, Load Balancing, and Replication - PostgreSQL Documentation - https://www.postgresql.org/docs/current/high-availability.html
3. Row Security Policies - PostgreSQL Documentation - https://www.postgresql.org/docs/current/ddl-rowsecurity.html

10 10. SQL vs. NoSQL & NewSQL Architectures

To ace a senior system design interview, you must know when *not* to use a traditional relational database. The choice between relational and non-relational databases fundamentally boils down to how they handle distributed data over a network.

10.1 10.1 The Scale-Up vs Scale-Out Dilemma

- **SQL (Relational):** Historically scales *vertically* (Scale-Up). When you need more performance, you buy a bigger server with more RAM, CPU, and faster NVMe drives. This is because enforcing strict relational rules (joining data, locking rows, ensuring consistency) across a network of 100 separate servers is incredibly slow due to network latency.

- **NoSQL (Non-Relational):** Designed to scale *horizontally* (Scale-Out). You buy 100 cheap commodity servers and distribute the data across them. To achieve this speed across a network, NoSQL databases (like MongoDB, Cassandra, or DynamoDB) abandon strict relational schemas, lack traditional joins, and often relax data consistency.

10.2 10.2 The CAP Theorem

When you distribute data across multiple servers (a distributed system), you are bound by a fundamental mathematical constraint.

> *[!TIP]* ***Heavy Concept: The CAP Theorem Analogy:*** *You are managing a bank with two branches (Node A and Node B), connected by a phone line.*
>
> ***Concrete Example:*** *A storm knocks out the phone line. The branches can no longer talk to each other. This is a* ***Partition (P)****. Now, a customer walks into Branch A to deposit $100. Because you cannot communicate with Branch B to update its ledger, you have a forced choice: - **Choice 1 (Availability):** You accept the deposit. The system is Available. But now, Branch A says the balance is $100, and Branch B says it's $0. You have lost Consistency. This is an **AP** system. - **Choice 2 (Consistency):** You refuse the deposit until the phone line is fixed, ensuring both branches always have the exact same balance. You have maintained Consistency, but the customer was rejected. You lost Availability. This is a **CP** system.*
>
> ***Plain English:*** *The CAP theorem states you can only guarantee two out of three: **C**onsistency (every read receives the most recent write or an error), **A**vailability (every request receives a non-error response), and **P**artition Tolerance (the system continues to operate despite an arbitrary number of messages being dropped by the network).*
>
> ***Technical:*** *Because network partitions (P) are unavoidable in distributed systems (switches fail, cables are cut), database engineers cannot actually choose CA. They must choose between **CP** (Consistency over Availability during a partition) and **AP** (Availability over Consistency during a partition).*
>
> ***The Trap:*** *Thinking a traditional single-node PostgreSQL database is "CA". While a single node provides Consistency and Availability, it is not a distributed system, so the theorem doesn't apply. The moment you distribute PostgreSQL (e.g., via synchronous replication across datacenters), it becomes a **CP** system: it will block writes if the network fails, preferring to go offline rather than corrupting consistency.*

10.3 10.3 ACID vs. BASE Semantics

Because NoSQL databases often choose Availability over Consistency (AP), they operate on **BASE** semantics instead of the strict **ACID** properties you learned for relational databases.

- **B**asically **A**vailable: The database guarantees a response to any request (success or failure), even if the data returned is slightly stale because a recent update hasn't propagated to all nodes yet.
- **S**oft State: The state of the system can change over time without any new input, simply due to background syncing between nodes.

- **E**ventually Consistent: If no new updates are made to a given piece of data, eventually all nodes will synchronize and return the exact same value.
  - *Example:* When you "like" a post on a global social network, the counter updates instantly for you, but your friend in another country might see the old count for a few seconds. For social media, this eventual consistency is fine; for a banking ledger, it is unacceptable.

10.4 10.4 The Rise of NewSQL and Distributed PostgreSQL

For decades, engineers faced a stark binary choice: use SQL for strict ACID transactions but hit a vertical scaling ceiling, or use NoSQL for infinite horizontal scale but lose transactions and joins.

NewSQL (also known as Distributed SQL) is a class of modern databases that attempts to solve this, providing the horizontal scalability and geographical distribution of NoSQL while maintaining strict ACID compliance and a standard SQL interface.

- **Purpose-built Distributed SQL:** Databases like Google Cloud Spanner and CockroachDB were built from the ground up for this. They achieve global consistency using advanced consensus protocols (like Raft) and specialized hardware (Spanner uses atomic clocks and GPS receivers to definitively order transactions globally).
- **PostgreSQL as Distributed SQL:** The PostgreSQL ecosystem has evolved to bridge this gap. Extensions like **Citus** transform a standard PostgreSQL database into a distributed database, automatically sharding tables and routing queries across a cluster of Postgres nodes while maintaining SQL semantics. This allows teams to scale horizontally without leaving the PostgreSQL dialect or ecosystem.

10.4.1 References

1. Scale Up vs Scale Out - PostgreSQL Wiki - https://wiki.postgresql.org/wiki/Scale-Out
2. CAP Theorem - IBM - https://www.ibm.com/topics/cap-theorem
3. ACID vs BASE - Medium - https://medium.com/swlh/acid-vs-base-in-databases-324c4dc8c1e7
4. Citus Data (Distributed PostgreSQL) - https://www.citusdata.com/

11 11. SQL Interview & OA Question Bank

This section contains classic SQL questions that appear repeatedly in technical interviews and online assessments (OAs) at companies like Amazon, Google, Meta, and top startups. Each problem is presented with the question, the schema, the solution, and an explanation of *why* this approach works. Problems are organized from foundational to advanced.

11.1 11.1 Foundational — SELECT, WHERE, Aggregates

11.1.1 Q1: Find employees who earn more than their manager

Schema: `employees (emp_id, name, salary, manager_id)`

```sql
SELECT e.name AS employee, e.salary AS emp_salary,
       m.name AS manager,  m.salary AS mgr_salary
FROM employees e
INNER JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Why: This is a classic **Self Join**. The same `employees` table is aliased as both `e` (the employee) and `m` (the manager). The `JOIN` condition links each employee's row directly to their manager's row, allowing a direct comparison of their salaries in the `WHERE` clause.

11.1.2 Q2: Find departments with more than 5 employees

Schema: `employees (emp_id, name, dept_id)`, `departments (id, dept_name)`

```sql
SELECT d.dept_name, COUNT(e.emp_id) AS headcount
FROM departments d
LEFT JOIN employees e ON d.id = e.dept_id
GROUP BY d.dept_name
HAVING COUNT(e.emp_id) > 5
ORDER BY headcount DESC;
```

Why: `HAVING` filters *after* aggregation, unlike `WHERE` which filters individual rows *before* grouping. We use a `LEFT JOIN` in case a department exists but has zero employees (though the `> 5` condition makes an `INNER JOIN` functionally identical here).

11.1.3 Q3: Find customers who have never placed an order

Schema: `customers (id, name)`, `orders (id, customer_id, order_date)`

```sql
-- Approach 1: LEFT JOIN + IS NULL (often the fastest execution plan)
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.id IS NULL;

-- Approach 2: NOT EXISTS (cleaner semantics, strictly NULL-safe)
SELECT c.name
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);
```

Why: Both approaches are valid PostgreSQL idioms. Approach 1 attempts to join every customer to an order, and the `WHERE o.id IS NULL` filters out any customer where a match was found. Approach 2 explicitly asks the database to check for the absence of rows, which is highly readable and immune to `NULL` comparison bugs that plague the `NOT IN` operator.

11.1.4 Q4: Find duplicate rows in a table

Schema: `contacts (id, email, name)`

```sql
-- Find which emails are duplicated
SELECT email, COUNT(*) AS cnt
FROM contacts
GROUP BY email
HAVING COUNT(*) > 1;

-- Find the actual duplicate rows with all their data
SELECT * FROM contacts
WHERE email IN (
    SELECT email FROM contacts GROUP BY email HAVING COUNT(*) > 1
);
```

Why: Aggregating by the column you suspect has duplicates (`email`) and filtering with `HAVING COUNT(*) > 1` immediately identifies the offending values. You can then use those values in a subquery to pull the full rows.

11.2 11.2 Intermediate — Joins, Subqueries, CASE

11.2.1 Q5: Second Highest Salary (Classic)

Schema: `employees (emp_id, name, salary)`

```sql
-- Approach 1: Window Function (Extensible to Nth highest)
SELECT salary AS second_highest FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk = 2;

-- Approach 2: Subquery with MAX (Simple, but rigid)
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Why: The `DENSE_RANK()` window function approach is superior because it handles ties flawlessly and scales easily if the interviewer changes the question to "5th highest". The `MAX()` subquery approach is clever but only works for the 2nd highest. (Note: Avoid using `LIMIT 1 OFFSET 1` for this problem, as it will skip the wrong row if two people tie for the highest salary).

11.2.2 Q6: Nth Highest Salary per Department

Schema: `employees (emp_id, name, salary, dept_id)`

```sql
-- Find the 3rd highest salary in each department
SELECT dept_id, name, salary FROM (
    SELECT dept_id, name, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
rnk
    FROM employees
) ranked
WHERE rnk = 3;
```

Why: `PARTITION BY dept_id` creates an independent ranking window for each department. `DENSE_RANK()` ensures that if two people tie for 1st place, the next person is ranked 2nd (unlike `RANK()`, which would skip to 3rd).

11.2.3 Q7: Top N records per group

Schema: `orders (id, customer_id, total, order_date)`

```sql
-- Top 3 most expensive orders per customer
SELECT customer_id, id, total, order_date FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY total
DESC) AS rn
    FROM orders
) ranked
WHERE rn <= 3;
```

Why: We use `ROW_NUMBER()` (instead of `RANK()` or `DENSE_RANK()`) because we want exactly 3 rows returned per customer, even if there are identical order totals that would cause ties in a ranking function.

11.2.4 Q8: Pivot — Rows to Columns

Schema: `sales (product, quarter, revenue)`

```sql
-- Convert quarterly rows into columns
SELECT product,
    SUM(CASE WHEN quarter = 'Q1' THEN revenue ELSE 0 END) AS Q1,
    SUM(CASE WHEN quarter = 'Q2' THEN revenue ELSE 0 END) AS Q2,
    SUM(CASE WHEN quarter = 'Q3' THEN revenue ELSE 0 END) AS Q3,
    SUM(CASE WHEN quarter = 'Q4' THEN revenue ELSE 0 END) AS Q4,
    SUM(revenue) AS total
FROM sales
GROUP BY product;
```

Why: This `SUM(CASE ...)` pattern is the standard SQL way to pivot data without relying on vendor-specific pivot extensions. It evaluates each row, places the revenue in the correct column based on the quarter, and aggregates them up to the product level.

11.2.5 Q9: Year-over-Year Growth

Schema: `revenue (year, month, amount)`

```
SELECT year, month, amount,
       LAG(amount) OVER (PARTITION BY month ORDER BY year) AS
prev_year_amount,
       ROUND(
           (amount - LAG(amount) OVER (PARTITION BY month ORDER BY year)) *
100.0
           / NULLIF(LAG(amount) OVER (PARTITION BY month ORDER BY year), 0),
       2) AS yoy_growth_pct
FROM revenue
ORDER BY month, year;
```

Why: `LAG()` looks "backwards" to fetch a value from a previous row within the same window (partitioned by month, ordered by year, so it looks at the same month in the prior year). `NULLIF` is a crucial safety check to prevent a division-by-zero crash if the previous year had exactly `0` revenue.

11.2.6 Q10: Consecutive Days Login (Gaps and Islands)

Schema: `logins (user_id, login_date)` (one row per login per day)

```
-- Find users who logged in for 3+ consecutive days
WITH numbered AS (
    SELECT user_id, login_date,
           login_date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY
login_date) * INTERVAL '1 day' AS grp
    FROM logins
)
SELECT user_id, MIN(login_date) AS streak_start, MAX(login_date) AS
streak_end,
       COUNT(*) AS streak_days
FROM numbered
GROUP BY user_id, grp
HAVING COUNT(*) >= 3
ORDER BY user_id, streak_start;
```

Why: This is the classic **Gaps and Islands** technique. If a user logs in on the 1st, 2nd, and 3rd of the month, `ROW_NUMBER()` generates 1, 2, and 3. Subtracting the row number (as an interval of days) from the actual date results in the exact same base date for all consecutive days. This base date (`grp`) acts as a unique identifier for that "island" of activity.

11.3 11.3 Advanced — Window Functions, CTEs, Complex Logic

11.3.1 Q11: Running Total and Moving Average

Schema: `transactions (id, account_id, txn_date, amount)`

```
SELECT account_id, txn_date, amount,
    SUM(amount) OVER (PARTITION BY account_id ORDER BY txn_date
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
running_total,
    AVG(amount) OVER (PARTITION BY account_id ORDER BY txn_date
                      ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
moving_avg_7day
FROM transactions
ORDER BY account_id, txn_date;
```

Why: The `ROWS BETWEEN` framing clause defines the explicit boundaries of the window. `UNBOUNDED PRECEDING` includes everything from the start of the partition up to the current row (a running sum). `6 PRECEDING` defines a rolling 7-day window (the current day + the 6 days prior).

11.3.2 Q12: Delete Duplicate Rows (Keep One Copy)

Schema: `contacts (id, email, name)` — `id` is a primary key, but `email` has duplicates.

```
WITH dupes AS (
    SELECT id,
           ROW_NUMBER() OVER (PARTITION BY email ORDER BY id ASC) AS rn
    FROM contacts
)
DELETE FROM contacts
WHERE id IN (SELECT id FROM dupes WHERE rn > 1);
```

Why: The CTE uses `ROW_NUMBER()` to assign an ascending integer to each email group. By ordering by `id ASC`, the oldest record gets `rn = 1`. Any row with `rn > 1` is a duplicate. The outer `DELETE` targets those duplicates by their unique `id`. This avoids complex self-joins and works flawlessly in PostgreSQL.

11.3.3 Q13: Median Salary

Schema: `employees (emp_id, salary)`

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary
FROM employees;
```

Why: PostgreSQL natively supports ordered-set aggregate functions. `PERCENTILE_CONT(0.5)` computes the continuous 50th percentile (the exact median), automatically interpolating between

two values if there is an even number of rows. This completely replaces the need for complex, manual row-counting logic.

11.3.4 Q14: Department with Highest Average Salary

Schema: `employees (emp_id, name, salary, dept_id)`, `departments (id, dept_name)`

```sql
SELECT dept_name, avg_salary FROM (
    SELECT d.dept_name, AVG(e.salary) AS avg_salary,
           RANK() OVER (ORDER BY AVG(e.salary) DESC) AS rnk
    FROM employees e
    JOIN departments d ON e.dept_id = d.id
    GROUP BY d.dept_name
) ranked
WHERE rnk = 1;
```

Why: We aggregate the average salary and apply a window function (`RANK()`) in the same subquery. Grouping happens first, so the window function evaluates the aggregated `AVG(salary)` over all departments. Filtering `rnk = 1` in the outer query safely handles ties if two departments share the exact top average.

11.3.5 Q15: Employees with Salary Above Department Average

Schema: `employees (emp_id, name, salary, dept_id)`

```sql
SELECT name, salary, dept_id, dept_avg FROM (
    SELECT name, salary, dept_id,
           AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
    FROM employees
) t
WHERE salary > dept_avg;
```

Why: Window functions compute values across rows without collapsing them (unlike `GROUP BY`). By using `AVG(salary) OVER (PARTITION BY dept_id)`, every employee row gets a column containing their department's average, allowing a direct row-level comparison in the outer query without needing a messy self-join.

11.3.6 Q16: Find Managers with at Least 5 Direct Reports

Schema: `employees (emp_id, name, manager_id)`

```sql
SELECT m.name AS manager, COUNT(e.emp_id) AS direct_reports
FROM employees e
INNER JOIN employees m ON e.manager_id = m.emp_id
GROUP BY m.emp_id, m.name
HAVING COUNT(e.emp_id) >= 5;
```

Why: We perform a self-join where the `e` table represents the direct reports and the `m` table represents the managers. We group by the manager's ID and filter out any managers who have fewer than 5 rows linked to them.

11.3.7 Q17: Cumulative Sum that Resets

Schema: `events (id, user_id, event_date, points)` — running sum must reset whenever points are negative.

```
WITH groups AS (
    SELECT *, SUM(CASE WHEN points < 0 THEN 1 ELSE 0 END)
              OVER (PARTITION BY user_id ORDER BY event_date) AS grp
    FROM events
)
SELECT user_id, event_date, points,
       SUM(points) OVER (PARTITION BY user_id, grp ORDER BY event_date) AS
running_points
FROM groups;
```

Why: This is a two-step window function problem. First, we create an artificial grouping ID (`grp`) by taking a running sum of a boolean condition (is it negative?). Every time a negative value appears, the `grp` ID increments. Second, we partition our actual running total by both the `user_id` *and* this new `grp` ID, effectively resetting the sum at every boundary.

11.3.8 Q18: Swap Odd and Even Rows

Schema: `seat (id, student)` — swap adjacent students: row 1↔2, 3↔4, etc.

```
SELECT
    CASE
        WHEN id % 2 = 1 AND id = (SELECT MAX(id) FROM seat) THEN id
        WHEN id % 2 = 1 THEN id + 1
        ELSE id - 1
    END AS id,
    student
FROM seat
ORDER BY id;
```

Why: We use a `CASE` statement to dynamically alter the `id` values in the result set. Odd IDs become even (`+ 1`), and even IDs become odd (`- 1`). The first `WHEN` clause is a safety check: if the total number of students is odd, the very last student has no one to swap with, so their ID must remain unchanged.

11.3.9 Q19: Tree Node Classification

Schema: `tree (id, p_id)` — `p_id` is the parent node ID; NULL means root.

```sql
SELECT id,
    CASE
        WHEN p_id IS NULL THEN 'Root'
        WHEN id IN (SELECT p_id FROM tree WHERE p_id IS NOT NULL) THEN
'Inner'
        ELSE 'Leaf'
    END AS type
FROM tree
ORDER BY id;
```

Why: A node is the Root if it has no parent. A node is an Inner node if it serves as a parent for at least one other node in the table. If it is neither, it is a Leaf. We use an `IN` subquery to check if the current node's `id` appears anywhere in the `p_id` column.

11.3.10 Q20: Consecutive Numbers

Schema: `logs (id, num)` — find numbers that appear at least 3 times consecutively.

```sql
SELECT DISTINCT num AS ConsecutiveNums FROM (
    SELECT num,
           LAG(num, 1) OVER (ORDER BY id) AS prev,
           LEAD(num, 1) OVER (ORDER BY id) AS next
    FROM logs
) t
WHERE num = prev AND num = next;
```

Why: `LAG(num, 1)` fetches the previous row's number, and `LEAD(num, 1)` fetches the next row's number. If the current row's number perfectly matches both the previous and the next, it must be the middle of a 3-row consecutive streak. `DISTINCT` ensures we only return the number once, even if it appeared 4 or 5 times in a row.

11.4 11.4 Rapid-Fire Theory Questions (Verbal Interview)

#	Question	Key Answer
1	WHERE vs HAVING?	`WHERE` filters rows before `GROUP BY` aggregates them; `HAVING` filters groups after aggregation.
2	DELETE vs TRUNCATE vs DROP?	`DELETE` removes row-by-row and can be rolled back. `TRUNCATE` rapidly deallocates the table's storage pages. `DROP` destroys the table schema entirely.
3	CHAR vs VARCHAR?	`CHAR` is fixed-length (pads with spaces). `VARCHAR` is variable-length (saves space).
4	Primary Key vs Unique Key?	A Primary Key guarantees uniqueness and `NOT NULL`, and a table can only have one. A Unique Key allows `NULL`s, and a table can have many.
5	UNION vs UNION ALL?	`UNION` deduplicates the combined results (slower). `UNION ALL` keeps all duplicates (faster).
6	Clustered vs Non-Clustered Index?	Clustered physically reorders the table data on disk (only 1 per table). Non-clustered is a separate b-tree pointer structure (many per table).
7	Correlated vs Non-Correlated Subquery?	A non-correlated subquery runs once. A correlated subquery references the outer query and must run row-by-row (often much slower).
8	RANK vs DENSE_RANK vs ROW_NUMBER?	Given ties: `ROW_NUMBER` assigns 1, 2, 3. `RANK` assigns 1, 1, 3 (leaves a gap). `DENSE_RANK` assigns 1, 1, 2 (no gap).
9	What makes a predicate non-SARGable?	Wrapping a column in a function, leading wildcard `LIKE '%...'`, implicit type conversion, or `OR` across different columns, which forces a table scan.

11.4.1 References

1. Top SQL Interview Questions - GeeksForGeeks - https://www.geeksforgeeks.org/sql-interview-questions/
2. LeetCode SQL Problems - LeetCode - https://leetcode.com/problemset/database/
3. PostgreSQL Window Functions - PostgreSQL Documentation - https://www.postgresql.org/docs/current/tutorial-window.html